

INF203 : PROGRAMMATION WEB

I. OBJECTIFS

✓ Objectif général :

Cette UE permet à l'étudiant de programmer des applications web.

✓ Objectifs spécifiques :

- Initier des apprenants à s'accoutumer avec des technologies web (protocoles, architectures, serveurs, navigateurs).
- Initier aux apprenants comment concevoir des sites web statique et dynamique.
- Donner la possibilité ou l'occasion aux apprenants d'avoir l'idée générale sur le webmastering et webdesign.
- Exposer comment passer de l'analyse à la conception dans la programmation Web avec l'utilisation de MySQL comme SGBD.
- Faire la programmation classique ou orientée objet avec des langages web.
- Etre capable de développer des applications web en fusionnant html, JavaScript, Css, PHP, Xml et autres.
- Initier les apprenants à travailler en équipe pour des projets.

II. CONTENU DU COURS.

Chapitre 1 : Notions sur les langages des balises.

Chapitre 2 : Langage de programmation du coté client : JavaScript.

Chapitre 3 : : Langage de programmation du coté serveur: PHP.

Chapitre 4 : La base de données avec MySQL+ PHP.

CHAPITRE I : NOTIONS SUR LES LANGAGES DES BALISES

I.1. Introduction

On appelle web (toile en français), contraction de « World Wide Web », une possibilité offerte par le réseau Internet de naviguer entre les documents reliés par de liens hypertexte. Le concept du web a été mis au point au CERN (Centre Européen de Recherche Nucléaire) en 1991 par une équipe de chercheur à laquelle appartenaient Tim Berner LEE, le créateur du concept d'hyperlien, considéré aujourd'hui comme père fondateur du Web **(Dallaire, 2004)**

Le principe du web repose sur l'utilisation d'hyperliens pour naviguer entre les documents (pages Web) grâce à un logiciel appelé navigateur (browser en anglais). Une page Web est un fichier informatique unique écrit en HTML et peut contenir de nombreux éléments multimédias tels que texte, des images, des sons, des animations, etc. et pour être à d'autres pages Web à l'aide de balises.

En dehors de liens reliant les pages Web, le Web prend tout son sens avec le protocole http permettant de lier des documents hébergés par des ordinateurs distants (appelés serveurs Web par opposition du client que représente le navigateur). Sur Internet les documents sont ainsi repérés par une adresse appelée URL, permettant de localiser une ressource sur n'importe quel serveur du réseau Internet.

Le web est sans doute une technologie majeure du 21ème siècle. Et si sa nature, sa structure et son utilisation ont évolué au cours du temps, force est de constater que cette évolution a également profondément modifié nos pratiques dans tout le plan.

L'activité des entreprises et administrations repose de plus en plus sur les technologies et applications Web. Les nouvelles applications sont aujourd'hui presque systématiquement développées avec des technologies

Web, et les anciennes applications sont souvent adaptées pour être accédées via un navigateur Web

I.2. Applications Web.

I.2.1. Définition.

Une application Web est un logiciel applicatif manipulable à l'aide d'un navigateur Web. Les applications Web analogiquement aux sites Web sont généralement placées sur un ordinateur jouant le rôle de serveur (Twitter, Facebook, etc. sont des exemples d'applications Web).

I.2.2. Protocoles web et notion de port.

A. Protocoles web

✓ Protocole IP (Internet Protocol)

Une adresse IP permet d'identifier de façon unique une machine sur un réseau. Cette adresse est sous la forme de quatre octets séparés chacun par un point. Chacun de ces octets appartient à une classe selon l'étendue du réseau.

Pour faciliter la compréhension humaine, un serveur particulier appelé DNS (**D**omaine **N**ame **S**ervice) est capable d'associer un nom à une adresse IP.

✓ Protocole TCP

TCP est un protocole orienté connexion qui assure le contrôle des erreurs. Lors d'une communication à travers ce protocole, les deux processus (**client** et **serveur** qui peuvent être distants ou locaux) doivent établir une connexion permanente entre eux ; cette connexion ne pourra être fermée que si l'un d'entre eux la ferme ou si un problème provoquant la fermeture de cette connexion surgit.

Ce protocole offre des performances moins bonnes (c'est le prix à payer pour la fiabilité) et utilise la notion de port pour permettre à plusieurs applications d'exploiter ce même protocole.

✓ Le protocole UDP

UDP est un protocole non orienté connexion n'assurant pas le contrôle des erreurs (il ne garantit pas que les données arrivent dans l'ordre

d'émission). Lors d'une communication à travers le protocole UDP, un processus X qui contacte un processus Y initie la communication mais la connexion établie entre les deux ne dure que le temps de la transmission des données (de X vers Y), elle est automatiquement fermée une fois le transfert terminé. Lorsque X aura besoin une seconde fois de contacter Y il devra donc établir une nouvelle connexion (La connexion est établie chaque fois que les deux processus souhaitent communiquer). Ce protocole offre de bonnes performances car il est très rapide.

Pour assurer les échanges, UDP utilise la notion de port, ce qui permet à plusieurs applications d'utiliser UDP sans que les échanges interfèrent les uns avec les autres. Cette notion est similaire à la notion de port utilisée par TCP.

✓ **Protocole HTTP.**

Le protocole de transfert hypertexte (HyperText Transfer Protocol) est le principal canal de diffusion de données sur Internet, principalement des fichiers HTML (mais également de tous les types de fichiers). Il dispose de nombreuses méthodes lui permettant théoriquement d'accomplir de nombreuses actions sur le serveur. En pratique, tous les serveurs ne les implémentent pas, ce qui limite l'usage de http à quelques méthodes clefs.

✓ **Protocole FTP**

Protocole de transfert de fichier (File Transfer Protocol), il a créé un flux de données entre le serveur et le client qui peut être beaucoup plus long que via http (celui-ci ferme la connexion dès que le document est envoyé). Il comprend bien plus des méthodes d'accès aux données que http, ce qui le rend bien plus utile pour le transfert de grand nombre de fichiers (vers ou depuis le serveur). A la différence de la plupart des protocoles, FTP utilise deux connexions au lieu d'une seule : L'une pour envoyer les commandes vers le serveur tout en recevant des informations, l'autre pour le transfert des données.

Protocole SMTP

Les protocoles de transports de courriers électroniques. SMTP en lui-même ne fait que transporter un message d'un point à un autre, en aucun cas il ne prend

en charge la structure de ce message. Il ne permet pas non d'accéder à une messagerie.

Protocole POP/ IMAP

Les protocoles POP et IMAP s'en chargent. POP et IMAP permettent d'accéder à un compte de messagerie et d'en tirer les messages. Là où POP récupère l'intégralité des messages vers le logiciel client, IMAP peut les laisser sur le serveur, les marquer comme lus, les classer... POP est donc utilisé pour la gestion de la boîte mail, tandis qu'IMAP permet une gestion complète et à distance

Protocole MIME.

Les paquets acheminent les données, mais rien ne permet vraiment de déterminer ce qu'ils contiennent. C'est le rôle de MIME (Multipurpose Internet Mail Extensions). Tout ce qui concerne l'encodage, le type de données ou le jeu de caractères est décrit dans l'en-tête MIME du message non textuel. À l'origine utilisé pour les mails, le protocole HTTP a récupéré sa syntaxe.

B. Notions de port.

De nombreuses applications réseaux pouvant être exécutées simultanément sur une même machine, il est nécessaire que celle-ci sache toutes les identifier. Pour ce faire, chacune de ces applications se voit attribuer une adresse unique codée sur 16 bits appelée port. Ainsi, l'utilisation de l'adresse IP associée à un port donne une identification unique à une application s'exécutant sur une machine se trouvant dans un réseau (ou pas). Un port peut être vu comme un point d'entrée à un service d'un processus se trouvant sur une machine connectée à un réseau (ou pas).

Il existe des milliers de ports, raison pour laquelle une assignation standard a été mise au point afin d'aider à la configuration des réseaux. Les ports 0 à 1023 sont les ports reconnus ou réservés. Ils sont assignés par l'IANA (Internet Assigned Number Authority) et sont, sur beaucoup de systèmes, uniquement utilisables par les processus systèmes ou par des programmes exécutés par des utilisateurs privilégiés. Les ports 1024 à 49151 sont appelés

ports enregistrés et les ports 49152 à 65535 sont les ports dynamiques ou privés.

I.2.3. Serveurs Web.

De nos jours, les serveurs d'applications comportent en leur sein un serveur Web (HTTP) qui est chargé d'écouter toutes les requêtes HTTP généralement sur le port 80. La structuration en couches des différents composants mis à disposition par le serveur d'application permet une prise en compte des besoins métier, des interactions avec les utilisateurs, des connexions avec les bases de données, etc.

On appelle serveur web tout logiciel permettant à des clients d'accéder à des pages web à partir du navigateur installé sur leur ordinateur distant. Il peut également être défini comme un "simple" logiciel capable d'interpréter les requêtes **HTTP** (HyperText Transfer Protocol) arrivant sur le port associé au protocole HTTP, et de fournir une réponse avec ce même protocole.

Voici quelques serveurs web les plus utilisés :

- ❖ Microsoft IIS (Internet Information Server);
- ❖ Apache ;
- ❖ Varnish.
- ❖ Apache Tomcat Coyote.
- ❖ BIG-IP.
- ❖ Rack Cache.
- ❖ Nginx.
- ❖ Phusion Passenger
- ❖ Microsoft PWS (Personal Web Server) ;
- ❖ Xitami.
- ❖ Microsoft ASP.NET
- ❖ GlassFish serveur d'application Open Source de Oracle
- ❖ Etc

a. Le client

Le client est généralement un navigateur Web à travers lequel des requêtes sont envoyées au serveur d'application via le protocole HTTP, HTTPS, FTP, etc. à l'aide d'une URL que l'on entre dans la barre d'adresses

Icones des quelques navigateurs les plus utilisés:



Microsoft Edge



Internet Explorer



Google Chrome



Mozilla Firefox



Opera

b. Les URL (Uniform Resource Locator)

C'est un format de nommage universel représenté sous forme de chaîne de caractère ASCII pour désigner une ressource sur le Web. Cette chaîne de caractère combine les informations nécessaires pour indiquer à un serveur d'application comment accéder à une ressource. Ces informations comprennent généralement: le protocole de communication, un nom de domaine, un numéro de port et un chemin d'accès.

Exemple : <http://www.exemple.com/chemin/ressource>

- **http** : Le protocole de communication (Ici HTTP)
- **www.exemple.com** : Nom de domaine du service représentant l'adresse IP et le port 80 par défaut de la machine sur laquelle s'exécute ce service (on peut utiliser l'adresse IP à la place du nom de domaine).
- **chemin/ressource** : nom complet de la ressource à demander sur le service une fois connecté. On peut aussi ajouter des données supplémentaires lors de la demande de la ressource. On appelle ces données des paramètres, elles sont transmises à la suite du caractère « ? » et séparées par le caractère &.

I.2.4. Architecture Web.

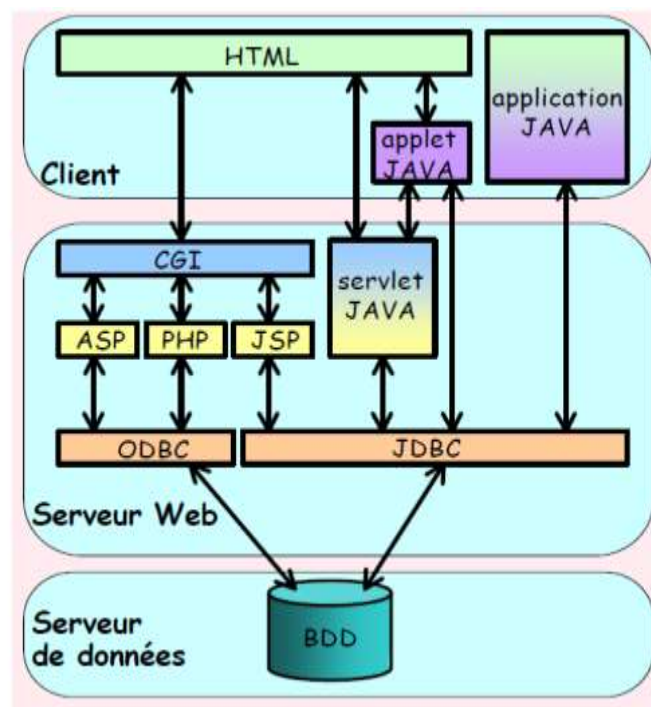
a. Document Web

Les documents échangés sur le Web peuvent être de types très divers. Le principal est le document hypertexte, un texte dans lequel certains mots, ou groupes de mots, sont des liens, ou ancres, donnant accès à d'autres documents.

b. Client Web

Le Web est donc un ensemble des serveurs connectés à l'Internet et proposant des ressources. L'utilisateur qui accède à des ressources utilise en général un type particulier de programme client, le navigateur. Les deux principales tâches d'un navigateur consistent à :

- Dialoguer avec un serveur ;
- Afficher à l'écran des documents transmis par un serveur.



Les navigateurs offrent des fonctionnalités bien plus étendues que les deux tâches citées ci-dessus. Netscape offre par exemple des modules permettant de composer des documents HTML. Généralement un document web est créé grâce à un langage de balisage (html, xhtml, ...).

De ce fait, nous distinguons 2(deux) types des clients :

☞ Client léger.

Le poste client accède à une application située sur un ordinateur dit « serveur » via une interface et un navigateur Web. L'application fonctionne entièrement sur le serveur, le poste client reçoit la réponse « toute faite » à la demande (requête) qu'il a formulée.

➔ Client lourd.

Le poste client doit comporter un système d'exploitation capable d'exécuter en local une partie des traitements. Le traitement de la réponse à la requête du client utilisateur va mettre en œuvre un travail combine entre l'ordinateur serveur et le poste client.

I.5. Notion de site web

I.5.1. Définition

Un **site web** (aussi appelé site internet) est un ensemble de fichiers HTML, liés par des liens hypertextes, stockés sur un serveur web, c'est-à-dire un ordinateur connecté en permanence à internet, hébergeant les pages web. Cette connexion est nécessaire enfin de les rendre accessibles par tous.

I.5.2. Catégories de site web.

En fonction des buts poursuivis, les sites web peuvent être regroupés en catégories suivantes :

- **Les sites personnels** : ce sont des sites réalisés par des particuliers. Ce genre des sites n'a souvent pas d'objectif visé. Ils juste à titre de loisir, par passion d'une discipline, d'un domaine, d'une star, etc.
- **Les sites communautaires** : ce sont des sites réunissant les utilisateurs autour d'un intérêt commun.
- **Les sites intranet** : ce sont des sites mis au point au sein d'une entreprise ou organisation en vue de permettre la communication entre différents services de l'entreprise pour l'échange des informations.
- **Les sites d'information** : leur but est de fournir des informations particulières à un type précis d'internautes et dans des domaines bien précis.
- **Les sites catalogues** : leur unique but est de présenter l'offre de l'entreprise ou de l'organisation.
- **Les sites vitrines** : ce sont des sites qui ont pour objectif la présentation de l'image de l'entreprise. Attendez par là la présentation des services offerts et

les produits vendus par cette dernière. Ils sont également appelés *sites plaquettes ou sites identités*.

- **Les sites marchands** : ce sont des sites permettant la vente des produits en ligne. Le paiement s'effectuant également de la même manière (c'est-à-dire en ligne).
- **Les sites institutionnels** : ce sont des sites qui présentent une organisation. On y décrit toutes les activités et on fournit toutes les organisations se rapportant à ladite entreprise.

I.5.3. Notion d'ergonomie.

A. Les couleurs dans une page web.

Il est conseillé de ne pas utiliser plus de trois couleurs différentes dans un site web afin de respecter le critère de sobriété. Le choix des couleurs devra correspondre, le cas échéant, aux couleurs de l'organisation (notamment aux couleurs du logo) et exprimer une ambiance particulière.

Quel que soit le choix des couleurs, il est recommandé d'établir une couleur prédominante, représentant la majeure partie de la page web, et une ou plusieurs couleurs secondaires plus dynamiques (plus vives), en moindres proportions, afin de mettre des éléments en exergue.

Les couleurs possèdent une symbolique implicite. Il est donc important de les choisir en connaissance de cause. En effet, les couleurs influent sur le comportement des individus :

- physiquement : appétit, sommeil, température du corps, etc.
- émotionnellement : sentiment de peur, de sécurité, de joie, etc.
- Psychologiquement : dynamisme, concentration, etc.

Voici un tableau récapitulant la signification classiquement associée aux couleurs :

Couleur	Symbolique méliorative	Symbolique péjorative	Domaine
---------	------------------------	-----------------------	---------

Bleu	Calme, confiance, autorisation, apaisement, sérénité, protection, sérieux, mystique, bonté, eau, espace, paix	Froid, sommeil	Voile, nouvelles technologies, informatique, médecine
Violet	Délicatesse, passion, discrétion, modestie, religion.	Mélancolie, tristesse, deuil, insatisfaction	Culture, politique
Rose	Charme, intimité, femme, beauté	Naïveté	Journal personnel, femmes
Rouge	Chaleur, force, courage, dynamisme, triomphe, amour, enthousiasme.	Violence, colère, danger, urgence, interdit, sang, enfer.	Luxe, mode, sport, marketing, médias
Orange	Tièdeur, confort, gloire, bonheur, richesse, honneur, plaisir, fruit, odeur, tonus, vitalité.	Feu, alerte	Divertissement, sport, voyage
Jaune	Lumière, gaieté, soleil, vie, pouvoir, dignité, or, richesse, immortalité.	Tromperie, égoïsme, jalousie, orgueil, alerte	Tourisme
Vert	Nature, vie végétale, secours, équilibre, foi, apaisement, repos, confiance, tolérance, espoir, orgueil, jeunesse, charité.		Découverte, nature, voyage, éducation
Brun	Calme, philosophie, terroir.	Saleté	Environnement
Blanc	Pureté, innocence, neige, propreté, fraîcheur, richesse.		Mode, actualités
Gris	Neutralité, respect.		Design, associations, organisations à but non lucratif

Noir	Sobriété, luxe, nuit.	Mort, obscurité, tristesse, monotonie.	Cinéma, art, photographie et interdit
-------------	-----------------------	--	---------------------------------------

Il s'avère important de signaler que le choix de la couleur d'arrière-plan (en anglais *background*) est primordial, car un arrière-plan mal choisi peut gêner la lisibilité. Un bon contraste entre la couleur d'avant-plan et la couleur dominante de l'arrière-plan est nécessaire. A ce titre, il est fortement déconseillé d'opter pour un arrière-plan graphique car il peut gêner la lecture et dégage généralement un sentiment d'amateurisme. L'arrière-plan devra ainsi généralement être choisi très pâle.

B. Choix de la police dans un site web.

Il est fortement contre-indiqué d'utiliser plus de deux polices sur un site web. Les polices stylisées doivent être utilisées avec parcimonie (pour le titre par exemple) et l'essentiel de la page web devra être réalisée avec une police classique (Arial, Verdana, Helvetica, etc.).

Pour une utilisation imprimée traditionnelle, les polices à empattement (serif) facilitent généralement la lecture car les empattements permettent d'accompagner le regard du lecteur.

Sur le web, l'utilisation de telles polices est déconseillée car selon la définition de l'écran du visiteur, les empattements peuvent très vite devenir des pattes de mouche gênantes pour la lecture. Ainsi, il est souhaitable d'opter pour des polices sans empattement (sans serif), plus rondes.

Enfin, sachez que les textes utilisant des polices non standard risquent de ne pas apparaître correctement sur les écrans des internautes. Pour créer des titres avec de telles polices, il est néanmoins possible de contourner cette limitation en créant des images transparentes comportant le texte.

I.5.4. Hébergement et référencement.

A. L'hébergement.

Pour rendre un site Internet accessible à tout instant, il faut l'amener sur serveur relié en permanence à l'Internet. La société offrant ce genre de service est appelé hébergeur ; et le service qu'elle fournit s'appelle l'hébergement. Les hébergeurs sont situés dans des salles d'hébergements spécialisés appelés **Data Center** (Centre des données).

Il existe deux sortes d'hébergement :

- **Les hébergeurs gratuits ;**
- **Les hébergeurs professionnels.**

Les hébergeurs gratuits prêtent gratuitement de l'espace sur un serveur. En revanche, l'hébergeur gagne de l'argent par la publicité effectuée sur le site ou par le trafic sur le site. Le service fourni par ce genre d'hébergeurs n'est pas de très bonne qualité.

Par contre les hébergeurs professionnels sont obtenus moyennant quelques frais. Ils offrent un service de qualité supérieure (bande passante) et assure la sécurité des données. Si le site a un trafic important et l'on désire obtenir un nom de domaine ; alors on est obligé d'avoir un hébergeur professionnelle. Pour ce genre de service, l'espace réservée à l'accueil des machines a une importance capitale.

Il existe trois sortes d'hébergement professionnel :

- **L'hébergement mutualisé** : quand le serveur héberge un grand nombre des sites ;
- **L'hébergement dédié** : quand un serveur est complètement loué par un entreprise ou une organisation.
- **La colocation** : quand une baie d'hébergement est louée en vue les serveurs du client. Une baie est armoire se trouvant dans de **Data**

Center, pouvant recevoir des éléments dans des emplacements ayant une largeur de dix-neuf pouces.

B. Le référencement

Le référencement désigne l'ensemble des techniques permettant d'améliorer la visibilité d'un site web. Parmi ces techniques nous pouvons citer :

- **indexation** : c'est l'opération qui consiste à faire connaître le site auprès des outils de recherche grâce aux formulaires qu'ils possèdent ;
- **positionnement** : c'est l'opération qui consiste à placer le site ou certaines pages de ce dernier en première page de résultat pour certains mots-clés ;
- **classement** : c'est une opération qui a un but identique au positionnement mais lui concerne des expressions plus élaborées.

Si l'on veut donner plus de référencement aux pages du site ; il faudra avoir un contenu attrayant, bien choisir le titre, avoir une adresse adaptée, un corps de texte lisible par les moteurs, les balises Meta (ce sont des balises insérées au début des documents HTML, mais qui ne s'affichent pas sur la page. Elles décrivent exactement le contenu de la page), des liens bien pensés et des attributs ALT pour déterminer le contenu des images.

C. Les différents noms des domaines.

Le nom de domaine d'un site est l'adresse ou l'url de celui-ci. Il est conseillé de choisir des noms de domaines simples car ce genre des noms de domaines peut facilement se transmettre de bouche à oreille. Il faudra donc éviter des noms de domaines longs ou compliqués.

Les noms de domaines sont repartis selon les activités des sites. Nous pouvons citer les noms suivants :

- ✓ **.sarpa** : pour les machines issues du réseau originel ;
- ✓ **.com** : attribué aux entreprises à vocation commerciale ;
- ✓ **.edu** : attribué aux organismes éducatifs ;
- ✓ **.gov** : attribué aux organismes gouvernementaux ;
- ✓ **.mil** : attribué aux organismes militaires ;
- ✓ **.net** : pour les organismes ayant trait aux réseaux ;

- ✓ **.org** : pour les entreprises à but non lucratif ;
- ✓ **.fr** : site situé dans la région française;
- ✓ **.ca** : site situé dans la région canadienne ;
- ✓ **.cd** : site situé dans la région congolaise ;
- ✓ **.sh** : site situé dans la région française.

I.5.5. Présentation générale d'une page web

Il convient de noter que la taille de la page web dépend essentiellement de la définition de l'affichage. En vue d'obtenir des pages plus attractives, il est nécessaire de respecter une certaine ergonomie. Le terme Ergonomie désigne l'utilisation de connaissances scientifiques dans le but d'améliorer son environnement de travail. Elle est caractérisée par l'efficacité (c'est-à-dire l'adoption des solutions appropriés d'utilisation d'un produit) ; la sécurité (le choix des solutions adéquats pour protéger l'utilisateur) et le confort d'utilisation (c'est-à-dire faire en sorte que l'utilisateur se fatigue le moins possible en lisant la page).

Puisque les utilisateurs sont de profils différents il faudra alors tenir compte des leurs attentes (tous les utilisateurs n'ont pas les mêmes attentes) ; leurs habitudes ; leurs âge ; les équipements (l'affichage du site dépend du navigateur et de la résolution d'affichage) et leurs niveaux de connaissances (tenir compte du fait que tous les utilisateurs du site ne sont des experts en informatique. L'ergonomie devra alors est conçu en fonction de l'utilisateur le moins expérimenté).

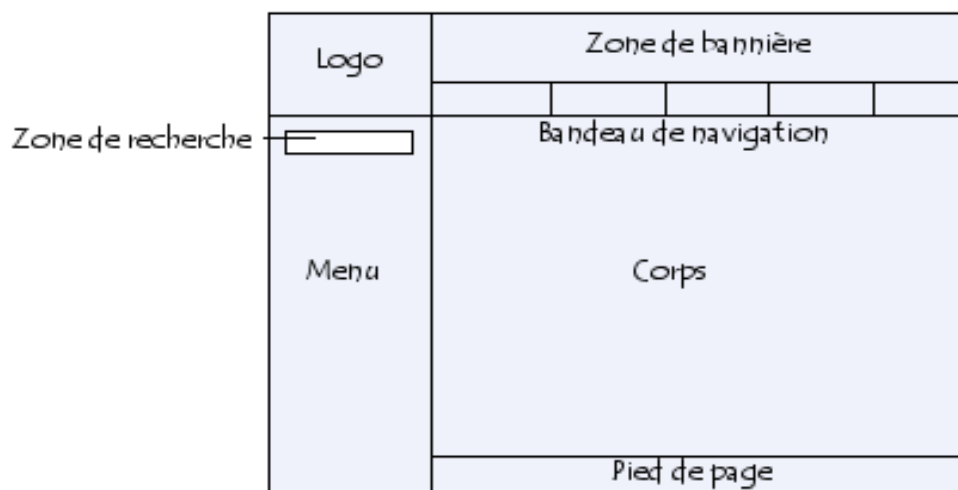
Aussi faut-il savoir que la position des informations d'une page web est d'une importance capitale. Ainsi les informations les plus importantes doivent être situées au début de la page.

Généralement les pages web contiennent les éléments ci-après :

- **Un en-tête** : il contient le nom du site, un bandeau de navigation et une zone réservée à la publicité ;

- **Un logo** : il est situé généralement au coin supérieur gauche de la page. Un clic sur ce logo nous amène à la page d'accueil. ;
- **Une zone de navigation** (également appelés menu) : elle située à gauche ou à droite de la page ;
- **Le corps de la page** : il contient tous les informations utiles.
- **Un pied de page** : il contient un lien vers un formulaire de contact, un plan d'accès, la date de la mise à jour et autres

Ainsi voici comment peut se présenter un site web :

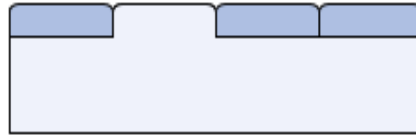


I.5.6. Les éléments de la navigation

Les éléments de navigation sont des outils présents sur une page web, permettant à l'utilisateur de se situer par rapport à l'ensemble du site et de parcourir toutes les rubriques du site. On peut mettre en œuvre les éléments de navigation par les moyens suivants :

- **Le fil d'Ariane** : c'est un outil de navigation comprenant une suite des liens ayant une hiérarchie. Il est caractérisé par des liens séparés par le caractère '>' (Accueil > Webmastering > **Navigation**). Le caractère '>' symbolise la hiérarchie. Le lien **Navigation** est non cliquable et représente la page sur laquelle se trouve l'internaute.

- **La navigation par onglets** : les onglets permettent de distinguer les rubriques et de les parcourir facilement. Elle se présente de la manière suivante :



- **La carte du site (ou plan du site)** : permet à l'utilisateur d'avoir un aperçu sur tout le contenu du site en un seul coup d'œil.

I.6. Le webmastering

L'ensemble des tâches utiles à la création d'un site web est désigné par le terme **webmastering**. Un **webmaster** est donc la personne se chargeant des tâches mentionnées ci-dessous. La création d'un site web est un projet à part entière qui nécessite un certain nombre des taches préalables à savoir :

- **La conception** : elle consiste à la détermination des objectifs du site, des personnes ciblées et de la structure du site. En vue d'obtenir une satisfaction générale, il est nécessaire d'associer quelques représentants de l'entreprise bénéficiaire du site. L'objectif principal de cette phase est d'analyser tous les besoins et d'imaginer les différents concepts d'utilisation du site.
- **La réalisation** : elle consiste à l'écriture des fichiers **HTML** en vue de la création des pages web. Ces fichiers **HTML** peuvent être introduits à la main (c'est-à-dire en saisissant le code dans un simple éditeur des textes) ou grâce un éditeur spécial (**WYSIWYG** : **W**hat **Y**ou **S**ee **I**s **W**hat **Y**ou **G**et). Le **WYSIWYG** est un logiciel permettant la création des pages web visuellement, juste en plaçant des objets et des contrôles. Le code HTML est automatiquement généré par Le logiciel. L'inconvénient de cette méthode est le fait qu'elle est très ennuyeuse si l'éditeur ne permet pas de réaliser exactement ce que souhaite l'utilisateur.

- **L'hébergement** : il consiste à amener son site web sur un serveur web, en vue de le rendre disponible sur Internet à tout moment qu'un utilisateur voudra y accéder.
- **La promotion et référencement** : la promotion est l'ensemble des activités qui permet que le site soit connu par une quantité suffisante des personnes. Par contre le référencement est l'ensemble des techniques qui permettent d'améliorer la visibilité d'un site web.
- **La maintenance et la mise à jour** : elles consistent à l'entretien du site et l'actualisation des informations contenues dans ce dernier.

I.7. Le webdesign.

Le terme « **webdesign** » désigne la discipline consistant à structurer les éléments graphiques d'un site web afin de traduire, à travers une dimension esthétique, l'identité visuelle de la société ou de l'organisation. Il s'agit ainsi d'une étape de **conception visuelle**, par opposition à la conception fonctionnelle (ergonomie, navigation).

L'objet du webdesign est de valoriser l'image de l'entreprise ou de l'organisation par le biais d'éléments graphiques afin de renforcer son identité visuelle et de procurer un sentiment de confiance à l'utilisateur. Néanmoins, en vertu des critères d'ergonomie, un site web doit avant tout répondre aux attentes des utilisateurs et lui permettre de trouver facilement l'information qu'il cherche.

Par extension le terme **webdesigner** désigne le métier consistant à concevoir le design d'un site web.

I.8. Extensible Hyper Text Markup Language « HTML ».

I.8.1. Structure d'une page html.

Une page HTML ou XHTML s'écrit de la façon suivante :

- ✓ Sur la première ligne, la balise `<!DOCTYPE ...>` indique la version de (X) HTML utilisée.

- ✓ Le reste de la page est encadré par des balises `<html>` et `</html>` qui signifient début et fin de HTML.
- ✓ Entre ces deux balises se trouvent deux parties : l'en-tête de la page entre `<head>` et `</head>` et le contenu (le corps) de la page entre `<body>` et `</body>`.

```
<!DOCTYPE ... >
<html>
<head>
<Title>Titre affiché dans la barre du navigateur
</title>
...
</head>
<body>...Contenu de la page... </body>
```

La balise `<!DOCTYPE ...>` étant longue et un peu barbare, elle n'est pas écrite ici en entier. Ne vous inquiétez pas pour autant : un simple copier-coller nous fournira le bon DOCTYPE et nous y reviendrons plus loin. Notez que cette balise est la seule à s'écrire en majuscules, toutes les autres sont en minuscules.

Délimité par les balises **`<head>`** et **`</head>`**, l'en-tête donne des informations qui ne seront pas visibles dans la page web, sauf la balise **`<title>`** qui fournit le titre de la page affiché dans la barre de titre, tout en haut de la fenêtre du navigateur. Les autres balises de l'en-tête indiquent la langue et le codage utilisés, les styles (feuilles de style CSS), etc. Nous les détaillerons plus loin également.

Tout le contenu visible dans le navigateur, le texte comme les liens vers les images, se trouve dans le corps de la page entre les balises **`<body>`** et **`</body>`**.

I.8.2. Les balises.

Les balises sont des éléments de base du HTML. Ils permettent la structuration des documents avec insertion des titres et des images, le réglage de la taille de

la police, etc. Ces balises sont toujours exprimées sous la forme d'un mot clé, encadré par les caractères "<" et ">". Exemple : <BALISE>. Pour la plupart des balises, il existe une balise de fermeture associée, reprenant le même nom, mais précédé du caractère "/". Exemple : </BALISE>. La commande spécifiée s'applique donc uniquement au texte situé entre le couple de balises ainsi formé.

Exemple:

```
<HTML>  
...  
</HTML>
```

Notons qu'une balise peut indifféremment être indiquée en minuscules ou en majuscules et le formatage "manuel" du document (espaces, sauts de lignes,...) est toujours ignoré.

I.8.3. Les attributs.

Un attribut est un élément, présent au sein de la balise ouvrante, permettant de définir des propriétés supplémentaires. Les attributs se présentent la plupart du temps comme une paire clé=valeur, mais certains attributs ne sont parfois définis que par la clé.

Voici un exemple d'attribut pour la balise <p> (balise définissant un paragraphe), permettant de spécifier que le texte doit être aligné sur la droite :

<p align="right">Exemple de paragraphe</p>

Chaque balise peut comporter un ou plusieurs attributs, chacun pouvant avoir aucune, une ou plusieurs valeurs.

I.8.4. Types de balises

Il existe deux sortes des balises :

- ✓ Les balises de structure ;
- ✓ Les balises de style.

I.8.4.1. Les balises de structure.

Comme le nom l'indique, les balises de structure permettent de structurer le document c'est-à-dire la sélection de la police, la présentation des paragraphes, le réglage de la taille de la police,...

Parmi les balises de structures nous pouvons citer :

- `<Hn>` pour le réglage de la taille des caractères ; où n= 1,2 ,3....6 ;
- `<P>` pour le passage à la ligne suivante ;
- `<BR>` ou `<BR/>` pour passer à la ligne suivante ;
- `<PRE>` pour conserver le formatage du texte encadré ;

Les attributs suivants peuvent être appliqués aux balises :

- **Align** : pour l'alignement du texte. Il peut avoir la valeur right, left ou center ;
- **Width** : pour régler la largeur. Ses valeurs sont soit des entiers soit des pourcentages ;
- **Size** : pour régler la taille. Il a pour valeurs des chiffres (ou nombres).

N.B : Il est possible d'insérer des éléments qui ne seront pas affichés sur la page, en utilisant des balises de commentaires. Ces éléments servent juste de référence au concepteur.

Exemple : `<!-- Commentaire -->`.

I.8.4.2. Les balises de style

Les balises de style modifient la typographie du texte. Elles peuvent être imbriquées dans d'autres balises de style de la même façon qu'on le ferait avec un traitement de texte. Voici une liste de balises de style reconnues par la plupart des navigateurs :

➤ **Les styles logiques sont les suivantes :**

`<EM>` texte `</EM>` met le texte généralement en italique.

`<STRONG>` texte `</STRONG>` met le texte généralement en gras.

`<CODE>` texte `</CODE>` pour l'utilisation d'une police à chasse fixe (encombrement des Caractères constant).

<SAMP> caractères </SAMP> séquence de caractères littéraux.
<KBD> saisie </KBD> pour mettre en évidence une saisie d'utilisateur.
<VAR> variable </VAR> pour indiquer le nom d'une variable.
<DFN> définition </DFN> pour mettre en évidence la 1ère utilisation d'un terme.
<CITE> citation </CITE> pour mettre en évidence une citation (généralement en italique).
<ADDRESS> adresse </ADDRESS> cette commande n'est pas utilisée pour une adresse postale. Cette commande est généralement utilisée pour indiquer l'auteur d'un document ainsi que le moyen de le contacter ou bien elle donne l'adresse du document. Elle est souvent placée en fin de document.
<BLOCKQUOTE> texte </BLOCKQUOTE> cette commande constitue avec le texte un texte avec retrait à gauche et à droite.
<BLINK> texte </BLINK> pour faire clignoter le texte.
<STRIKE> texte barré </STRIKE> cette commande permet de barrer du texte.
<LISTING> code informatique </LISTING> cette commande permet d'afficher du code informatique
<BIG> texte </BIG> cette commande permet d'écrire un texte en plus gros caractères que les caractères en cours.
<SMALL> texte </SMALL> cette commande permet d'écrire un texte en plus petits caractères que les caractères en cours.
^{texte} cette commande permet d'écrire un texte en exposant.
_{texte} cette commande permet d'écrire un texte en indice.

➤ **Les styles physiques sont les suivantes :**

<I> texte </I> met le texte en italique.
 texte met le texte en gras.
<TT> texte </TT> pour l'utilisation d'une police à chasse fixe (encombrement des caractères constant).
<U> texte </U> souligne le texte.
<S> texte </S> barre le texte.
_{texte} indice.
^{texte} exposant.

I.8.5. Structure d'une page html

Un document HTML commence par la balise <HTML> et finit par la balise </HTML>. Il contient également un en-tête décrivant le titre de la page, puis un corps dans lequel se trouve le contenu de la page. L'en-tête est délimité par les balises <HEAD> et </HEAD>. Le corps est délimité par les balises <BODY> et </BODY>.

Dans l'en-tête, on ne met généralement qu'une seule information, le titre du document qui sera affiché en haut de la fenêtre du navigateur et qui apparaît dans les bookmarks (listes d'URL gérées par un navigateur, une sorte d'annuaire). Ce titre est indiqué entre les balises <TITLE> et </TITLE>.

I.8.6. Liste d'éléments

Une liste est un paragraphe structuré contenant une suite d'articles. Le langage HTML définit trois types de listes :

- La liste ordonnée ;
- La liste non ordonnée ;
- La liste de définition.

I.8.6.1. Les listes ordonnées.

Elles se présentent sous la forme suivante :

 Élément 1

 Élément 2

 Élément 3

A l'affichage on aura :

Élément 1

Élément 2

Élément 3

- ✓ La liste ordonnée est encadrée par les balises et ;
- ✓ Les éléments de la liste sont encadrés les balises et ;
- ✓ La balise peut avoir un attribut nommé« type » ;
- ✓ L'attribut 'type' peut avoir pour valeurs :
 - 1 : numérotation chiffrée (par défaut) ;
 - A : numérotation en lettres majuscules ;
 - a : numérotation en lettres minuscules ;
 - I : numérotation en chiffres romains majuscules (I, II, III, IV ...) ;
 - i : numérotation en chiffres romains minuscules.

I.8.6.2. Les listes non ordonnées

Elles se présentent sous la forme suivante :

```
<ul>
  <li> Elément 1  </li>
  <li> Elément 2  </li>
  <li> Elément 3  </li>
</ul>
```

A l'affichage on aura :

Elément 1

Elément 2

Elément 3

- ✓ La liste non ordonnée est encadrée par les balises et ;
- ✓ Les éléments de la liste sont encadrés les balises et ;
- ✓ La balise peut avoir un attribut nommé« type » ;
- ✓ L'attribut type peut avoir pour valeurs :
 - Circle : puce circulaire ;
 - Square : puce carrée ;
 - Disc : puce en disque.

I.8.6.3. Les listes de définition

Elles se présentent sous la forme suivante :


```
<dl>
  <dt> Terme1  </dt>
    <dd> Définition du Terme1  </dd>
  <dt> Terme2  </dt>
    <dd> Définition du Terme2  </dd>
  <dt> Terme3  </dt>
    <dd> Définition du Terme3  </dd>
</dl>
```

A l’affichage on aura :

Terme1
Définition du Terme1 ;
Terme2
Définition du Terme2 ;
Terme3
Définition du Terme3.

- ✓ La liste de définition est encadrée par les balises <dl> et </dl> ;
- ✓ Les éléments de la liste sont encadrés les balises <dt> et </dt> ;
- ✓ Les définitions des éléments de la liste sont encadrés les balises <dd> et </dd>

Hormis l’attribut TYPE, les listes peuvent aussi avoir les attributs suivants :

- COMPACT : pour resserrer l'interligne ;
- PLAIN : pour supprimer les puces ;
- SEQNUM : pour définir le premier numéro ;
- START : pour déterminer le premier numéro.
- CONTINUE : repart du numéro où il s’était arrêté à la liste précédente.

I.8.7. Les tableaux

Les tableaux sont des suites de lignes permettant de présenter des informations d’une manière mieux structurée qu’avec les listes. Une cellule d’un tableau peut contenir un texte, des listes, des images, des liens hypertextes ou encore des éléments de formulaire.

Généralement, les tableaux se présentent sous la forme suivante :

```
<TABLE >
<CAPTION> Titre du tableau </CAPTION>
<TR>
    <TH> Titre 1 </TH>
<TH> Titre 2 </TH>
<TH> Titre 3 </TH>
<TH> Titre 4 </TH>
</TR>
<TR>
<TH> Titre 5 </TH>
<TD> Donnée 1 </TD>
    <TD> Donnée 2 </TD>
<TD> Donnée 3 </TD>
</TR>
</TABLE>
```

On aura le résultat suivant:

Titre du tableau

Titre 1	Titre 2	Titre 3	Titre 4
Titre5	Donnée 1	Donnée 2	Donnée 3

- ✓ Le tableau est encadré par les balises <TABLE> et </TABLE>.
- ✓ La balise <TABLE> peut avoir les attributs suivants :
 - BORDER : pour régler la taille de la bordure. La valeur qui lui est attribuée est un entier positif.
 - BGCOLOR : permet de spécifier la couleur de fond d'un tableau ;
 - WIDTH : permet de définir la largeur qu'occupera le tableau dans la fenêtre du browser. Il est exprimé en pourcentage ;
 - CELLPADDING : permet de définir l'espacement entre le contenu
Des cellules et son encadrement. Sa valeur est un entier positif.

- CELSPACING : permet de régler l'épaisseur de la grille intérieure du tableau. Sa valeur est aussi un entier positif.
 - FLOAT : Spécifie la position du texte qui suivra la balise </TABLE>. On peut lui attribuer la valeur right ou left ;
 - COLS : pour spécifier le nombre de colonnes. Sa valeur est aussi un entier positif ;
 - FRAME : pour le contrôle les éléments individuels d'encadrement du tableau. Ses valeurs possibles sont NONE (aucun), TOP (au-dessus), BOTTOM (en bas), TOPBOT (tout en haut), SIDES (sur les côtés) et ALL (tous) ;
 - RULES : permet le contrôle les éléments de la grille des cellules. Ses valeurs sont NONE (aucun), BASIC (basique), ROWS (ligne), COLS (colonne), ALL (tous) ;
-
- ✓ Le titre du tableau est compris entre les balises <CAPTION> et </CAPTION> ;
 - ✓ La balise <CAPTION> peut avoir l'attribut ALIGN. Ce dernier permet de définir la position du titre du tableau. Ses valeurs sont TOP (au-dessus) et BOTTOM (en dessous) ;
 - ✓ Chaque ligne du tableau est encadré par les balises <TR> et </TR> (Table Row = ligne du tableau) ;
 - ✓ La balise <TR> peut avoir les attributs suivants :
 - VALIGN : pour obtenir un alignement du texte dans le sens vertical de la cellule. Sa valeur sont soit TOP (au-dessus) ; MIDDLE (au milieu) ; BOTTOM (en dessous) ;
 - ALIGN : pour obtenir un alignement du texte dans le sens horizontal de la cellule. Sa valeur est soit CENTER (centre) ; LEFT (gauche) ; RIGHT (droite) ; JUSTIFY (justifié) ;
 - BGCOLOR : permet de spécifier la couleur de fond d'une ligne du tableau.
-
- ✓ Les cellules d'en-tête d'un tableau sont encadrées par les balises<TH> et </TH> (Table Header = En-tête du tableau) ;
 - ✓ La balise <TH> peut avoir les attributs suivants :

- COLSPAN : permet de définir une cellule dont la largeur est un multiple de la colonne de base ;
 - ROWSPAN : permet de définir une cellule dont la hauteur est un multiple de la ligne de base ;
 - NOWRAP : pour interdire au le retour à la ligne. Tout le texte sera alors inscrit sur une ligne ;
 - VALIGN : idem que la balise <TR> ;
 - ALIGN : idem que la balise < TR>.
- ✓ Chaque cellule du tableau est encadrée par les balises <TD> et </TD> (Table Data = Donnée du tableau) ;
- ✓ La balise <TD> peut avoir les attributs suivants :
- COLSPAN : idem que la balise <TH>;
 - ROWSPAN : idem que la balise <TH>;
 - NOWRAP : idem que la balise <TH>;
 - VALIGN : idem que la balise <TR> ;
 - ALIGN : idem que la balise <TR> ;
 - BGCOLOR : permet de spécifier la couleur de fond d'une cellule du tableau.
- ✓ Quand un attribut d'alignement est appliqué à la balise <TR>, il s'applique sur toutes pour toutes les cellules de cette ligne ; à moins de définir un autre type d'alignement pour une balise <TD> de la ligne.

I.8.8. Les images.

L'utilisation des images dans un site web est importante car elle permet de rendre celui-ci plus attractif. Il faut cependant éviter l'usage excessif des images, au risque d'augmenter le temps de chargement du site et de nuire à sa lisibilité.

En HTML, les images sont insérées par la balise . Les attributs de cette balise sont suivants :

- SRC : permet déterminer l'url de l'image. Cet attribut est obligatoire ;
- ALT : permet de définir un texte alternatif qui s'affichera au cas où l'image spécifiée ne s'afficherait pas. Sa valeur un est une chaîne des caractères;

- TITLE : pour définir un texte qui s'affichera quand on survole l'image. Sa valeur est une chaîne de caractères ;
- ALIGN : pour définir l'alignement de l'image. Il peut avoir la valeur bottom (au-dessous), center (centre), left (gauche), middle (milieu), top (au-dessus), ou right (droite) ;
- HEIGHT : pour spécifier la hauteur de l'image. Il est exprimé en pixels ou en pourcentage ;
- WIDTH : pour spécifier la largeur de l'image. Il est exprimé en pixels ou en pourcentage ;
- LOWSRC : permet de définir une image alternative (généralement plus petite), affichée le temps que la vraie image soit chargée par le navigateur. Sa valeur est un url ;
- NAME : Permet de définir un nom pour l'image. Cet attribut sert à la gestion des images en JavaScript ;
- LONGDESC : renvoie à un url donnant la description de l'image ;
- HSPACE : pour régler l'ajustement entre l'image et le texte adjacent. Sa valeur est un entier positif ;
- VSPACE : définit l'ajustement entre l'image et le texte. Il est exprimé en pixels ;
- BORDER : pour définir la taille de la bordure de l'image. Il est exprimé en pixels.

Dans l'insertion des images il faut également spécifier l'extension de cette image. Cette extension est spécifiée juste après le nom de l'image dans l'attribut SRC.

Une image peut avoir les extensions suivantes :

- JPEG (.JPG) : avec cette extension, les images ayant un grand nombre de couleurs seront bien compressées, ce qui permettra donc d'avoir un temps de chargement moindre ;
- PNG : elle permet d'avoir des images qui s'affichent progressivement avec une profondeur de couleurs de 24 bits et des images dont on définit une couleur comme transparente.

- GIF : les images ayant cette extension ont presque les mêmes caractéristiques que les images PNG, sauf que l'extension GIF n'est limitée qu'à 256 couleurs maximum et que ce format n'est pas totalement libre.

I.8.9. Les liens hypertextes

Les liens hypertextes sont également appelés ANCRAGE. Ce sont des éléments de la page web permettant de basculer vers un autre endroit juste en cliquant dessus.

En cliquant sur un ancrage on peut aller vers un autre endroit du document ou vers un fichier HTML situé à un emplacement différent sur la machine qui héberge la page ou encore vers une autre machine.

Une ancre de départ est une zone active sur laquelle, l'utilisateur va cliquer pour faire apparaître une nouvelle page ; tandis qu'une ancre d'arrivée est une zone inactive spécifiant le point d'arrivée d'un lien.

On parle de lien externe quand le clic dessus amène vers une autre page tandis le lien local est celui rendant vers fichier situé sur le même ordinateur.

Pour créer des liens on se sert uniquement de la paire `<A>` et `</A>`. La balise `<A>` peut avoir les attributs suivants :

- HREF : pour spécifier l'endroit où l'on veut se rendre ;
- TARGET : pour préciser si l'on veut ouvrir un nouvelle fenêtre ou rester sur la même fenêtre en cliquant sur le lien. Ses valeurs sont respectivement `_BLANK` et `_TOP` ;
- ALINK : pour spécifier la couleur du lien actif ;
- VLINK : pour spécifier la couleur du lien déjà utilisé.

I.8.10. Les formulaires

Les formulaires sont des moyens permettant aux auteurs des pages web de naviguer avec les différents lecteurs. Dans les formulaires on réserve des zones dans lesquels l'utilisateur va introduire ses données. Chacune de ces zones est identifiée par un nom spécifique. Les données transmises dans ce formulaire

sont envoyées à l'adresse spécifiée l'utilisateur appui sur le bouton d'envoi. Il convient de noter que le formulaire utilise les tableaux pour sa présentation.

- ✓ Le formulaire est encadré par les balises <FORM> et </FORM>
- ✓ La balise <FORM> peut contenir les attributs suivants :
 - METHOD : indique la forme sous laquelle seront envoyées les réponses. Ses valeurs sont POST (envoi de données stockées dans le corps de la requête) ou GET (envoi des données codées dans l'URL, et séparées de l'adresse du script par un point d'interrogation) ;
 - ACTION : c'est ici qu'est précisé l'adresse où seront envoyées les données du formulaire (script CGI ou adresse email (mailto:adresse.email@machine));
 - ENCTYPE : permet de spécifier le codage des données transmis. Cet attribut est facultatif ;
 - NAME : permet de spécifier le formulaire transmis lorsqu'il y a plus d'un formulaire dans le même site.
 - TARGET : permet d'indiquer le cadre ou la fenêtre où s'affiche la nouvelle page à l'exécution du script CGI.

I.8.10.1. La balise INPUT

La balise <INPUT> est d'une importance capitale dans le formulaire. En effet, elle permet la création des éléments interactifs. Sa syntaxe est la suivante :

<INPUT TYPE="Nom du champ" VALUE="Valeur par défaut" NAME="Nom de l'élément">

- L'attribut TYPE peut avoir les valeurs suivantes :
 - TEXT : un champ permettant la saisie des textes. On peut définir sa taille en lui ajoutant l'attribut Size, tandis que l'attribut Maxlength permet de fixer un maximum des caractères à saisir ;
 - CHECKBOX : c'est une case à cocher. Ses valeurs sont Checked (coché) et Unchecked (non coché) ;
 - HIDDEN : c'est un champ caché. Il est non visible sur le formulaire. Il permet de préciser un paramètre fixe qui sera envoyé au CGI ;

- **FILE** : champ permettant à l'utilisateur de préciser l'emplacement d'un fichier qui sera envoyé avec le formulaire. Il faudra alors préciser le type de données pouvant être envoyées grâce à l'attribut **ACCEPT** de la balise **<FORM>** ;
- **SUBMIT** : il permet l'envoi du formulaire. Il est appelé 'bouton de soumission'. Le texte du bouton est précisé par l'attribut **VALUE** ;
- **IMAGE** : c'est un bouton de soumission dont l'apparence est l'image située à l'emplacement précisé par son attribut **SRC** ;
- **RADIO** : il permet de faire un choix parmi plusieurs éléments proposés (l'ensemble des boutons radios doivent pour se faire porter le même attribut **NAME**. L'attribut **CHECKED** permet de préciser le bouton sélectionné par défaut ;
- **RESET** : c'est un bouton permettant de rétablir l'ensemble des éléments du formulaire à leurs valeurs par défaut.
- **PASSWORD** : c'est un champ de saisie, dans lequel les caractères saisis apparaissent sous forme d'astérisques.
 - L'attribut **NAME** : permet de reconnaître la donnée et la valeur qui a été saisie ;
 - L'attribut **VALUE** : permet d'affecter un texte par défaut dans le champ d'entrée des caractères.

I.8.10.2. La balise TEXTAREA

Cette balise permet de définir une zone de saisie plus vaste de la balise **INPUT**. Elle possède les attributs suivants:

- **COLS** : pour fixer le nombre de colonnes c'est-à-dire le nombre des caractères que peut contenir une ligne ;
- **ROWS** : pour fixer le nombre de lignes,
- **READONLY** : permet de rendre un champ en lecture seule ;
- **NAME** : permet d'associer un nom spécifique au champ ;
- **VALUE** : représente la valeur qui sera envoyée par défaut au script si le champ de saisie n'est pas modifié par l'utilisateur ;

I.8.10.3. La balise **SELECT**

Elle permet de créer une liste déroulante d'éléments. Ces éléments sont précisés par des balises **OPTION** se trouvant à l'intérieur de la balise **SELECT**. La balise **SELECT** peut avoir les attributs suivants :

- **NAME** : c'est le nom associé au champ, il permet d'identifier le champ dans la paire nom/valeur ;
- **SIZE** : il représente le nombre de lignes de la liste. Il peut avoir une valeur supérieure au nombre d'éléments présents dans la liste ;
- **DISABLE** : permet de créer une liste désactivée c'est-à-dire une liste inaccessible. Elle est affichée en gris ;
- **MULTIPLE** : permet à l'utilisateur de choisir plusieurs éléments au même moment de la liste.

I.8.11. Liste des nouvelles balises **HTML5**.

Les balises	La description
<area>	Définit une zone à l'intérieur d'une image
<article> (Nouvelle)	Définit un article
<aside> (Nouvelle)	Définit une partie latérale au contenu
<audio> (Nouvelle)	Définit un fichier audio
<bdo>	Définit la direction du texte
<button> (Nouvelle)	Définit un bouton cliquable
<canvas> (Nouvelle)	Définit un graphique
<col>	Définit une colonne d'un tableau
<colgroup>	Définit un groupe de colonne pour un tableau
<command> (Nouvelle)	Définit un bouton de commande
	Définit un texte supprimé
<details> (Nouvelle)	Définit les détails d'un élément
<embed> (Nouvelle)	Définit un contenu extérieur (audio, vidéo ...)
<figcaption> (Nouvelle)	Légende d'un groupe d'élément multimédia
<figure> (Nouvelle)	Définit un groupe d'élément multimédia
<footer> (Nouvelle)	Définit le bas d'une section ou d'une page
<header> (Nouvelle)	Définit le haut d'une section ou d'une page
<hgroup> (Nouvelle)	Regroupe les informations du haut d'une page ou section
<ins>	Définit un texte insérer
<keygen> (Nouvelle)	Générateur de clé pour un formulaire

<main>(Nouvelle)	Représente le contenu principal d'une page ou application.
<map>	Définit une carte
<mark>(Nouvelle)	Mise en valeur d'un mot ou d'un texte – Texte marqué
<math>(Nouvelle)	Définit une formule mathématique
<menu>	Définit un menu en liste
<meter>(Nouvelle)	Définit une unité de mesure
<nav>(Nouvelle)	Définit un groupe de liens de navigation
<noscript>	Définit une alternative au script non supporté
<optgroup>	Regroupe d'une liste d'option dans un formulaire
<output>(Nouvelle)	Définit un type de sortie
<progress>(Nouvelle)	Définit une progression
<rp>(Nouvelle)	Annotation ruby si le script n'est pas supporté
<rt>(Nouvelle)	Annotation ruby d'explication
<section>(Nouvelle)	Définit une section
<source>(Nouvelle)	Définit la source d'un contenu multimédia
	Définit une section de type inline
<summary>(Nouvelle)	Définit l'en-tête des détails d'un document ou section
<svg>(Nouvelle)	Définit une image vectorielle
<time>(Nouvelle)	Définit une unité de temps
<track>(Nouvelle)	Définit une unité de temps pour les éléments <audio> et <video>.
<video>(Nouvelle)	Définit une vidéo
<wbr>(Nouvelle)	Définit un possible retour à la ligne

I.9. Cascading Style Sheet (CSS).

I.9.1. Introduction.

On peut écrire du code CSS à trois endroits différents qui sont :

- ✓ Méthode A : Dans un fichier.css (le moyen le plus recommandé)
- ✓ Méthode B : Dans l'en-tête du fichier Xhtml
- ✓ Méthode C: Dans les balises

Voici la description de chacune de ces techniques. Si vous ne devez en retenir qu'une, retenez la première :

1. Dans un fichier. CSS (méthode A)

Comme je viens de vous dire, le plus souvent on écrit du code css dans un fichier spécial ayant l'extension.css (contrairement au fichier xhtml ou html qui ont l'extension. html)

✓ Ouvrir une page css et enregistre cette page avec l'extension. css

Ex : page : style. Css

✓ Enregistre cette page "style css " , et mettre dans l'en-tête<head> < /head>

Code HTML

```
< Link href = "nom de fichier. css " rel ="stylesheet" type ="text /css">
```

La balise < link/> comporte plusieurs attributs. Vous pouvez en modifier d'entre eux pour le moment :

- title : c'est le nom que vous donnez à votre feuille de style (mettez ce que vous voulez)
- href : c'est l'emplacement votre ou se trouve votre feuille de style sous forme de lien relatif ;

a2. Directement dans le header du fichier xhtml (méthode B)

Il est aussi possible de taper du css directement à l'intérieur même du fichier XHTML, entre les balises<head>< /head>.Vous devez y mettre une balise <style>< /style> à l'intérieur, comme ceci :

Code HTML

```
<Html>  
< Head>  
<Title>.....< /title>  
<style type="text/css">.....< /style>  
< /head>
```

3. La méthode "à l'arrache" dans les balises (méthode C)

Ça c'est la troisième (et la moins recommandée) des méthodes. Elle consiste à ajouter un attribut style sur les balises pour leur appliquer un style particulier. Cet attribut fonctionne sur toutes les balises.

Ex : **Code HTML**

```
< h1 style="propriété"> Texte </h1>
```

NB : Voici le commentaire en CSS : /* commentaire */

A) Appliquer en style a des balises.

➤ Dans un css, on trouve trois éléments différents :

- 1) Des noms de balises : on écrit les noms des balises dont on veut modifier l'apparence.
- 2) Des propriétés css : les effets de style de la page sont rangés dans des propriétés.
- 3) Les valeurs : à chaque propriété CSS on doit indiquer une valeur.

Schématiquement, une feuille de style CSS ressemble à ça :

Code CSS

```
1. Balise1
2. {
3.   Propriété : valeur ;
4.   Propriété : valeur ;
5.   Propriété : valeur ;
6. }
7. Balise2
8. {
9.   Propriété : valeur ;
10.  Propriété : valeur ;
11.  Propriété : valeur ;
12. }
13. Balise3
14. {
15.  Propriété : valeur ;
16. }
```

Exemple 1: Code CSS :

1. .P
2. {
3. Color : blue ;
4. Font-size : 18px ;
5. }

Interprétation : Cela signifie que : « je veux que tous mes paragraphes soient écrits en bleu avec une taille de 18pixels. »

Exemple 2 : code CSS

1. h1
2. {
3. Color :red ;
4. }
5. H2
6. {
7. Color :red ;
8. }

Interprétation : Nos balises h1 doivent être en rouge, ainsi que nos balises h2.

✓ Si les styles de 2 balises doivent avoir la même présentation. Il suffit de séparer les noms des balises par des virgules, comme ceci :

Ex : code CSS :

1. h1, h2
2. {
3. Color :red ;
4. }

Interprétation : « Je veux que le texte de mes titres < h1> et < h2> soient écrits en rouge »

I.9.2. Voici quelques propriétés utilisées.

Une application Web est un logiciel applicatif manipulable à l'aide d'un navigateur Web. Les applications Web analogiquement aux sites Web sont généralement placées sur un ordinateur jouant le rôle de serveur (Twitter, Facebook, etc. sont des exemples d'applications Web).

Propriétés	Interprétation	Valeurs	Exemples
<i>Font-family</i>	<i>Choix d'une police de caractères</i>		<i>arial, verdana</i>
<i>Font-size</i>	<i>Taille de police des caractères</i>		<i>h1 {font-size : 15% OU 15px ;}</i>
<i>Color</i>	<i>pour la couleur du texte</i>		<i>body {color : #0000ff ;}</i>
<i>Font-weight</i>	<i>préciser l'épaisseur d'écriture c.à.d. le degré de graissage de la police</i>	Normal : (valeur par défaut) bold : gras Lighter : moins gras que le style en cours Bolder : plus gras que le style en cours	<i>principale {font-weight : bold ;}</i>
<i>Font-style</i>	<i>mettre le texte en italique</i>	Normale : écriture droite (valeur par défaut), italic ; italique Oblique : police de type oblique	<i>remarque {font-style : italic ;}</i>
<i>Text-decoration</i>	<i>soulignement et autres « décoration »</i>	None : suppression du soulignement de liens au passage du souris Underline overline : titres de niveau 1 soulignement et surligne Line-through : barré Blink : clignotement	<i>h1 {text-decoration: underline overline ;}</i>

		<i>du texte</i>	
<i>Text-transform</i>	<i>mettre des caractères en majuscule ou minuscules</i>	Capitalize : première lettre de chaque mot en majuscule Lowercase : tout en minuscules Uppercase : tout en majuscules None : écriture standard (valeur par défaut)	<i>#pays {text-transform : uppercase ;}</i>
<i>Font – variant</i>	<i>mettre l’écriture en petite majuscules c.à.d. à peu près la taille des minuscules</i>	Normal : texte normal (valeur par défaut) Small –caps : tout en petites majuscules	<i>ville {font-variant : small – caps ;}</i>
<i>Background – color</i>	<i>Attribuer un fond de couleur à certaines lettres ou à certains mots, à la manière du surligneur sur papier. Cette propriété permettra et servira aussi à choisir la couleur de font d’un paragraphe ou d’un bloc.</i>	Transparent (valeur par défaut) : la couleur blanche.	<i>Strong { background-color }</i>
<i>Verticale-align</i>	<i>décalage vertical de lettres ou de mots, applicable aux cellules d’un tableau</i>	baseline : sur la base de la ligne (valeur par défaut) sub : indice super : exposant middle : au milieu de la ligne (centrage vertical sur la ligne) text-top ou text-bottom : alignement avec le haut ou le bas de la boîte parent	<i>-exposant, vertical-align : super}</i>

		valeur ou pourcentage : valeur positive pour décalage vers le haut, négative pour un décalage vers les bas	
Font	le raccourci permettant d'indiquer en une seule propriété, les mises en forme qui concernent l'italique, les petites majuscules, le graissage, la taille et la police de caractères	font-style, font-variant, font-weight, font-size, line-height : (hauteur de ligne, voir plus loin) et font – family.	<code>h3 {font : bold 1.2em verdana, sans-serif ;}</code>
Text-align	Aligner le texte	Left : Aligner à gauche (par défaut) Right : Aligner à droite Center : centrer Justify : justifié	<code>auteur {text-align : right ;}</code>

I.9.3. Utilisation des class et id.

Question:

Comment faire pour que seulement certains paragraphes (titres, citations) soient écrits d'une manière différente ?

Réponse:

On a la possibilité pour faire cela d'utiliser des attributs spéciaux qui fonctionnent sur toutes les balises :

- L'attribut class.
- L'attribut id.

Nous allons voir les classes suivantes dans l'ordre :

1. Ce que sont les class et id.
2. Les balises dites « Univers ».

3. Les imbrications de balises.

NB : Les attributs class et id sont quasiment identiques. Il y a seulement une petite différence que nous dévoilerons dans la suite.

a) Class

➤ C'est un attribut que l'on peut mettre sur n'importe quelle balise, aussi bien que paragraphe, image, titre, etc...

- `<h1 class= « »>.....</h1>`
- `<p class= « »>.....</p>`
- `<img class= « » />`

➤ Les class permettent de définir un style personnalisé.

Ex : Supposons que vous vouliez définir un style appelé « important » qui écrive le texte en rouge/18 pixels. Vous ajouterez l'attribut class= « important » à chacune des balises que vous voulez modifier.

Code CSS

```
1. Important
2 {
3 color: red;
4 font-size: 18 px ;
5 }
```

b) Id

- L'attribut id fonctionne exactement de la même manière que class, à un détail près : Il ne peut être utilisé qu'une fois.
- Un nom d'id doit commencer par une lettre et peut être suivi de lettres et de chiffres.

Avantage :

- il y en a assez peu pour tout vous dire, cela vous sera utile si vous faites du JavaScript plus tard pour reconnaître certaines balises.

- En pratique, nous ne mettrons des id que sur des éléments qui sont uniques sur votre page, comme par exemple le logo : `<img src = « images/logo.jpg » >`
- Dans le css, ce n'est pas un point que vous devez mettre le nom de l'id mais un dièse (#).

Code : CSS

1. #logo
2. {
3. /* Mettez les propriétés css ici */
4. }

Exercice

Reproduisez des balises correspondantes pour des formulaires ci-dessous :

VOICI LE FORMULAIRE D'INSCRIPTION

I. IDENTITE DU CANDIDAT

Numero	
Nom	
Postnom	
Prénom	
Date de naissance	01 / 01 / 1960
Sexe	MASCULIN
Etat-civil	Marié(e)
Nationalité	
Province	
Adresse	
Téléphone	

II. ETUDES SECONDAIRES FAITES

Nom de l'école secondaire fréquentée	
Adresse de l'école	
Nom du centre d'Examens d'Etat	
Année d'obtention de diplôme	1960
Pourcentage	%
Numero du diplôme :	

Valider

Supprimer

Modifier

ENREGISTREMENT DES AGENTS

Matricule :	24		
Nom :	kanyinda	Postnom :	kayembe
Prénom :	kams		
Sexe :	☒ Masculin ☐ Féminin		
Option :	Mathématicien		
Couleur préférée :	☒ Blanche ☐ Verte ☐ Bleue ☐ Rouge ☐ Jaune ☐ Orange		
Votre Age :	14		
	kitoko		
Commentaire :			

Valider

Effacer

Quitter

CHAPITRE II : PROGRAMMATION DU COTE CLIENT : JAVASCRIPT

II.1. Généralités

JavaScript est un langage de scripts qui incorporé aux balises Html, permet d'améliorer la présentation et l'interactivité des pages Web.

JavaScript est donc une extension du code Html des pages Web. Les scripts, qui s'ajoutent ici aux balises Html, peuvent en quelque sorte être comparés aux macros d'un traitement de texte. Ces scripts vont être gérés et exécutés par le browser lui-même sans devoir faire appel aux ressources du serveur. Ces instructions seront donc traitées en direct et surtout sans retard par le navigateur.

II.2. JavaScript n'est pas Java

Il importe de savoir que JavaScript est totalement différent de Java. Bien que les deux soient utilisés pour créer des pages Web évoluées, bien que les deux reprennent le terme Java (café en américain), nous avons là deux outils informatiques bien différents.

JavaScript	Java
Code intégré dans la page Html	Module (applet) distinct de la page Html
Code interprété par le browser au moment de l'exécution	Code source compilé avant son exécution
Codes de programmation simples mais pour des applications limitées	Langage de programmation beaucoup plus complexe mais plus performant
Permet d'accéder aux objets du navigateur Confidentialité des codes nulle (code source visible)	N'accède pas aux objets du navigateur Sécurité (code source compilé)

II.3. Le JavaScript minimum

a) La balise <SCRIPT>

De ce qui précède, vous savez déjà que votre script vient s'ajouter à votre page Web. Le langage Html utilise des tags ou balises pour "dire" au browser

d'afficher une portion de texte en gras, en italique, etc. Dans la logique du langage Html, il faut donc signaler au browser par une balise, que ce qui suit est un script et que c'est du JavaScript (et non du VB Script). C'est la balise :

`<SCRIPT LANGUAGE="JavaScript">..... </SCRIPT>`. Ou encore

`<SCRIPT TYPE="text/JavaScript">..... </SCRIPT>`

b) Les commentaires.

Il vous sera peut-être utile d'inclure des commentaires personnels dans vos codes JavaScript. C'est même vivement recommandé comme pour tous les langages de programmation (mais qui le fait vraiment ?). JavaScript utilise les conventions utilisées en C et C++ : soit `//ceci est un commentaire` ou soit `/* ceci est un commentaire */`.

c) Où inclure le code en JavaScript ?

Le principe est simple. Il suffit de respecter les deux principes suivants :

- ✓ n'importe où.
- ✓ mais là où il le faut.

Le browser traite votre page Html de haut en bas (y compris vos ajoutes en JavaScript). Par conséquent, toute instruction ne pourra être exécutée que si le browser possède à ce moment précis tous les éléments nécessaires à son exécution. Ceux-ci doivent donc être déclarés avant ou au plus tard lors de l'instruction. Pour s'assurer que le programme script est chargé dans la page et prêt à fonctionner à toute intervention de votre visiteur (il y a des impatients) on prendra l'habitude de déclarer systématiquement (lorsque cela sera possible) un maximum d'éléments dans les balises d'entête soit entre `<HEAD>` et `</HEAD>` et avant la balise `<BODY>`. Ce sera le cas par exemple pour les fonctions.

Rien n'interdit de mettre plusieurs scripts dans une même page Html. Il faut noter que l'usage de la balise `script` n'est pas toujours obligatoire. Ce sera le cas des événements JavaScript (par exemple `onClick`) où il faut simplement insérer le code à l'intérieur de la commande

Html comme un attribut de celle-ci. L'événement fera appel à la fonction JavaScript lorsque la commande Html sera activée.

JavaScript fonctionne alors en quelque sorte comme une extension du langage Html.

d) Une première instruction JavaScript.

Sans vraiment entrer dans les détails, voyons une première instruction JavaScript (en fait une méthode de l'objet Windows) soit l'instruction **alert ()**.

Syntaxe : alert ("votre texte");

Cette instruction affiche un message (dans le cas présent votre texte entre les guillemets) dans une boîte de dialogue pourvue d'un bouton OK. Pour continuer dans la page, le lecteur devra cliquer ce bouton.

II.4. Les événements

II.4.1. Généralités

Avec les événements et surtout leur gestion, nous abordons le côté "magique" de JavaScript. En Html classique, il y a un événement que vous connaissez bien. C'est le clic de la souris sur un lien pour vous transporter sur une autre page Web. Hélas, c'est à peu près le seul. Heureusement, JavaScript va en ajouter une bonne dizaine, pour votre plus grand plaisir. Les événements JavaScript, associés aux fonctions, aux méthodes et aux formulaires, ouvrent grand la porte pour une réelle interactivité de vos pages.

II.4.2. Le évènements

Evénement	Description
Click	Lorsque l'utilisateur clique sur un bouton, un lien ou tout autre élément.
Load	Lorsque la page est chargée par le browser ou le navigateur.
Unload	Lorsque l'utilisateur quitte la page
MouseOver	Lorsque l'utilisateur place le pointeur de la souris sur un lien ou tout autre élément
MouseOut	Lorsque le pointeur de la souris quitte un lien ou tout autre élément.
Focus	Lorsqu'un élément de formulaire a le focus c.-à-d. devient la zone d'entrée active.
Blur	Lorsqu'un élément de formulaire perd le focus c.-à-d. que

	l'utilisateur clique hors du champ et que la zone d'entrée n'est plus active.
Change	Lorsque la valeur d'un champ de formulaire est modifiée.
Select	Lorsque l'utilisateur sélectionne un champ dans un élément de formulaire
Submit	Lorsque l'utilisateur clique sur le bouton Submit pour envoyer un formulaire

II.4.3. Les gestionnaires d'évènements

Pour être efficace, il faut qu'à ces événements soient associées les actions prévues par vous. C'est le rôle des gestionnaires d'évènements.

La syntaxe est : on_ événement="fonction()"

Par exemple, onClick="alert('Vous avez cliqué sur cet élément')". De façon littéraire, au clic de l'utilisateur, ouvrir une boîte d'alerte avec le message indiqué.

II.4.3.1 OnClick.

Événement classique en informatique, le clic de la souris. Le code de ceci est :

```
<form>
  <input TYPE="button" VALUE="Cliquez ici" onClick="alert('Vous avez bien cliqué ici')">
</form>
```

Nous reviendrons en détail sur les formulaires dans le chapitre suivant.

II.4.3.2. OnLoad et OnUnload.

L'événement Load survient lorsque la page a fini de se charger. A l'inverse, Unload survient lorsque l'utilisateur quitte la page.

Les événements onLoad et onUnload sont utilisés sous forme d'attributs de la balise <BODY> ou <FRAMESET>. On peut ainsi écrire un script pour souhaiter la bienvenue à l'ouverture d'une page et un petit mot d'au revoir au moment de quitter celle-ci.

```

<html>
  <head>
    <title>JavaScript et HTML</title>
    <meta charset="UTF-8">
    <script LANGUAGE='Javascript'>
      function bienvenue() {
        alert("Bienvenue à cette page");
      }
      function au_revoir() {
        alert("Au revoir");
      }
    </script>
  </head>
  <body onLoad='bienvenue()' onUnload='au_revoir() '>
    HTML simple
  </body>
</html>

```

II.4.3.4. Onmouseover et Onmouseout

L'événement `onmouseover` se produit lorsque le pointeur de la souris passe au-dessus (sans cliquer) d'un lien ou d'une image. Cet événement est fort pratique pour, par exemple, afficher des explications soit dans la barre de statut soit avec une petite fenêtre genre infobulle.

L'événement `onmouseout`, généralement associé à un `onmouseover`, se produit lorsque le pointeur quitte la zone sensible (lien ou image).

Notons que si `onmouseover` est du JavaScript 1.0, `onmouseout` est du JavaScript 1.1. En clair, `onmouseout` ne fonctionne pas avec Netscape 2.0 et Explorer 3.0.

II.4.3.5. OnFocus

L'événement `onFocus` survient lorsqu'un champ de saisie a le focus c.-à-d. quand son emplacement est prêt à recevoir ce que l'utilisateur a l'intention de taper au clavier. C'est souvent la conséquence d'un clic de souris ou de l'usage de la touche "Tab".

II.4.3.6. OnBlur

L'événement onBlur a lieu lorsqu'un champ de formulaire perd le focus. Cela se produit quand l'utilisateur ayant terminé la saisie qu'il effectuait dans une case, clique en dehors du champ ou utilise la touche "Tab" pour passer à un champ. Cet événement sera souvent utilisé pour vérifier la saisie d'un formulaire.

Exemple :

```
<form>
  <input TYPE=text onBlur="alert('Ceci est un événement Blur')">
</form>
```

II.4.3.7. OnChange

Cet événement s'apparente à l'événement onBlur mais avec une petite différence. Non seulement la case du formulaire doit avoir perdu le focus mais aussi son contenu doit avoir été modifié par l'utilisateur.

II.4.3.8. OnSelect.

Cet événement se produit lorsque l'utilisateur a sélectionné (mis en surbrillance ou en vidéo inverse) tout ou partie d'une zone de texte dans une zone de type text ou textarea.

II.4.4. Gestionnaires d'événement disponibles en JavaScript.

Il nous semble utile dans cette partie "avancée" de présenter la liste des objets auxquels correspondent des gestionnaires d'événement bien déterminés.

Objets Gestionnaires d'événement disponibles :

<i>Objet</i>	<i>Événement</i>
Fenêtre	onLoad, onUnload
Lien hypertexte	onClick, onMouseOver, onMouseOut
Élément de texte	onBlur, onChange, onFocus, onSelect
Élément de zone de texte	onBlur, onChange, onFocus, onSelect
Élément bouton	OnClick
Case à cocher	OnClick
Bouton Radio	OnClick

Liste de sélection	onBlur, onChange, onFocus
Bouton Submit	OnClick
Button Reset	OnClick

II.4.5 La syntaxe d'Onmouseover.

Le code du gestionnaire d'événement onmouseover s'ajoute aux balises de lien : `<a href="" onmouseover="action ()">lien</a>`

Ainsi, lorsque l'utilisateur passe avec sa souris sur le lien, la fonction action () est appelée. L'attribut HREF est indispensable. Il peut contenir l'adresse d'une page Web si vous souhaitez que le lien soit actif ou simplement des guillemets si aucun lien actif n'est prévu. Nous reviendrons ci-après sur certains désagréments du codage href="".

Voici un exemple. Par le survol du lien "Compliment", une fenêtre d'alerte s'ouvre.

Le code est :

```
<body>
  <a href="" onmouseover="alert('Salut les amis')">Compliment</a>
</body>
```

On peut aussi utiliser ceci dans la balise <head>

```
<html>
  <head>
    <title>JavaScript et HTML</title>
    <meta charset="UTF-8">
    <script LANGUAGE='Javascript'>
      function salutation() {
        alert('Salut les amis');
      }
    </script>
  </head>
  <body>
    <a href="" onmouseover="salutation()">Compliment</a>
  </body>
</html>
```

II.4.6. La syntaxe d'Onmouseout.

Tout à fait similaire à onMouseOver, sauf que l'événement se produit lorsque le pointeur de la souris quitte le lien ou la zone sensible.

Au risque de nous répéter, si onMouseOver est du JavaScript 1.0 et sera donc reconnu par tous les browsers,

On peut imaginer le code suivant :

```
<a href="" onMouseOver="alert ('Salut les amis')"
onmouseout="alert ('A plus !')">Compliment </a>
```

Les puristes devront donc prévoir une version différente selon les versions JavaScript.

II.4.7 Problème! Et si on clique quand même...

Vous avez codé votre instruction Onmouseover avec le lien fictif `<a href=""...`, vous avez même prévu un petit texte, demandant gentiment à l'utilisateur de ne pas cliquer sur le lien et comme de bien entendu celui-ci clique quand même. Horreur, le browser affiche alors l'entièreté des répertoires de sa machine ou de votre site). Ce qui est un résultat non désiré et pour le moins imprévu. Pour éviter cela, prenez l'habitude de mettre l'adresse de la page encours ou plus simplement le signe # (pour un ancrage) entre les guillemets de HREF. Ainsi, si le lecteur clique quand même sur le lien, au pire, la page encours sera simplement rechargée et sans perte de temps car elle est déjà dans le cache du navigateur.

Prenez donc l'habitude de mettre le code suivant :

```
<a href="#" onMouseOver="action()">texte du lien</a>
```

II.5. Fonction et procédure

La fonction est un sous-programme qui permet d'effectuer un ensemble d'instruction par simple appel de son nom dans le corps du programme principal. Les fonctions et les procédures permettent d'exécuter dans plusieurs parties du programme une série d'instructions, cela permet une simplicité du code et donc une taille de programme minimale. Dans JavaScript les fonctions et les procédures sont définies par le mot *function*.

la différence entre une fonction et une procédure est que la fonction retourne une valeur (numérique, booléenne, etc...) ce qui n'est pas le cas pour une procédure. Ce retour de valeur se fait par le mot clé *return*.

Avant d'être utilisée, une fonction doit être définie pour que le navigateur puisse la connaître. La définition d'une fonction s'appelle « déclaration ».

La syntaxe pour déclarer une fonction :

```
function(param1, param3, ..., paramN)
{
    instructions_contenant_la_fonction;
}
```

Notez que les paramètres (arguments) sont optionnels. Cela dépend de ce que l'on veut faire.

Exemple :

```

<html>
  <head>
    <title>JavaScript</title>
    <meta charset="UTF-8">
    <script>
      function somme(un, deux)//Définition de la fonction
      {
        return un + deux;
      }

      function sommation()//Appel de la fonction dans une autre fonction
      {
        un = parseInt(document.getElementById('un').value);
        deux = parseInt(document.getElementById('deux').value);
        document.getElementById('reponse').value=somme(un, deux);
      }
    </script>
  </head>
  <body onload="somme()">
    <input type="text" id="un" placeholder="Premier nombre"/>
    <input type="text" id="deux" placeholder="Deuxieme nombre"/>
    <input type="button" value="Somme" onclick="sommation()"/><!--Appel de la fonction-->
    <input type="text" id="reponse" placeholder="Réponse"/>
  </body>
</html>

```

II.6. Méthodes

Une méthode est une fonction associée à un objet, c'est-à-dire une action que l'on peut faire exécuter à un objet. Les méthodes des objets des navigateurs sont des fonctions définies à l'avance par les normes HTML, on ne peut donc pas les modifier.

Par exemple une page HTML, est composée d'un objet appelé document. L'objet document a par exemple une méthode write() qui lui est associée et qui permet de modifier le contenu de la page HTML en affichant du texte. Une méthode s'appelle un peu comme une propriété, c'est-à-dire de la manière suivante : **window.objet.methode ()**

Exemple :

```
<html>
  <head>
    <title>Javascript</title>
    <meta charset="UTF-8">
    <script>
      function ouvre()
      {
        window.document.write("Bonjour le monde");
      }
    </script>
  </head>
  <body onload="ouvre()">

  </body>
</html>
```

Cet exemple affiche le texte « Bonjour le monde » sur la page du navigateur.

Un autre exemple d'impression d'une page web :

```
<html>
  <head>
    <title>Javascript</title>
    <meta charset="UTF-8">
    <script>
      function Impression()
      {
        window.print();
      }
    </script>
  </head>
  <body>
    <input type="button" value="Imprimer" onclick="Impression()" />
  </body>
</html>
```

II.7. Les structures de contrôle

Les structures de contrôle permettent de contrôler la séquence d'exécution d'un programme.

II.7.1. Les structures alternatives

Les structures alternatives test une certaine condition avant d'exécution ou non une série d'instruction.

II.7.1.1. Si (if)

La structure if (si) pour syntaxe suivante :

```
if (condition_vraie)
{
    instructions;
}
```

Exemple :

```

<html>
  <head>
    <title>JavaScript et HTML</title>
    <meta charset="UTF-8">
    <script LANGUAGE='Javascript'>
      function TestAge()
      {
        var age = document.getElementById('age').value;
        if (age > 18)
        {
          alert("Vous êtes majeur");
        }
      }
    </script>
  </head>
  <body>
    <form>
      <input type="text" id="age" placeholder="Votre âge"/>
      <input type="button" value="Test" onclick="TestAge()" />
    </form>
  </body>
</html>

```

Avec la structure **if** on peut l'écrire sous différentes versions :

a. If tout court

On recourt à la syntaxe précédente.

b. If...esle :

Voici la syntaxe à utiliser :

```

if (condition1) {
  instruction1;
}else {
  autre_instruction;
}

```

Exemple :


```
function TestAge()
{
    var age = document.getElementById('age').value;
    if (age > 18)
    {
        alert("Vous êtes accepté");
    }else{
        alert("Désolé! Vous n'êtes admis");
    }
}
```

c. if...else if...else

```
if (condition1)
{
    instruction1;
} else if (condition2)
{
    instruction2;
    ...
} else if (conditionN) {
    instructionN;
} else {
    autre_instruction;
}
```

Exemple :

```

<html>
  <head>
    <title>JavaScript et HTML</title>
    <meta charset="UTF-8">
    <script LANGUAGE='Javascript'>
      function VoirJourPair()
      {
        var jour = document.getElementById('age').value;
        if (jour ==2) { alert("Mardi. Jour pair");
        } else if(jour==4) { alert("Jeudi. Jour pair");
        }else if(jour==6){ alert("Samedi. Jour pair");
        }else{ alert("Désolé! On ne travaille que les jours pairs");
        }
      }
    </script>
  </head>
  <body>
    <form>
      <input type="text" id="age" placeholder="Votre âge"/>
      <input type="button" value="Test" onclick="VoirJourPair() ()"/>
    </form>
  </body>
</html>

```

Des toutes ces différentes manières de (**if**), si après le bloc de condition il y a une seule instruction à exécuter les accolades ne sont pas nécessaires.

L'exemple précédent peut alors s'écrire comme ceci :

```
<script LANGUAGE='Javascript'>
    function VoirJourPair()
    {
        var jour = document.getElementById('age').value;
        if (jour ==2)
            alert("Mardi. Jour pair");
        else if(jour==4)
            alert("Jeudi. Jour pair");
        else if(jour==6)
            alert("Samedi. Jour pair");
        else
            alert("Désolé! On ne travaille que les jours pairs");
    }
</script>
```

Grâce aux opérateurs logiques, on peut tester plus d'une condition avant d'exécuter une série d'instructions.

Exemple :

```

<html>
  <head>
    <title>HTML & JavaScript</title>
    <meta charset="UTF-8">
    <script>
      function Test()
      {
        var sexe = document.getElementById('sexe').value;
        var age = parseInt(document.getElementById('age').value);
        if ((sexe == 'M' || sexe == 'm') && (age <= 18))
          alert("Bonjour le petit");
        else if ((sexe == 'F' || sexe == 'f') && (age <= 18))
          alert("Bonjour la petite");
        else
          alert("On ne veut que des ados ici");
      }
    </script>
  </head>
  <body>
    <form>
      <input type="text" id="sexe" placeholder="Votre sexe SVP"/>
      <input type="text" id="age" placeholder="Votre age SVP"/>
      <input type="button" value="Test" onclick="Test()" />
    </form>
  </body>
</html>

```

d. le choix avec conditions imbriquées

On peut faire des tests imbriqués avant d'exécuter une série d'instructions. Voici la formulation des conditions imbriquées :

```

if (cond1) {
  if (cond1_1) {
    action
  }
} else if (cond2) {
  if (cond2_2) {
    action
  }
} else {
  action
}

```

Exemple :

```
<script>
function Test()
{
    var sexe = document.getElementById('sexe').value;
    var age = parseInt(document.getElementById('age').value);
    if (sexe == 'M' || sexe == 'm')
    {
        if (age <= 18)
            alert("Bonjour le petit");
    } else if (sexe == 'F' || sexe == 'f') {
        if (age <= 18)
            alert("Bonjour la petite");
    } else
        alert("On ne veut que des ados ici");
}
</script>
```

II.7.1.2. Switch

Cette structure permet de faire des tests sur une expression équivalent à la structure if...else if...else à la seule différence près que cette dernière pointe directement là où la condition est satisfaisante, contrairement à la structure précédente qui parcourt l'ensemble de cas testé.

Le switch a comme syntaxe :

```
var expres;
switch(expres)
{
    case valeur1:
        instruct1;
        break;
    case valeur2:
        instruct2;
        break;

    case valeurN:
        instructN
        break;
    default :
        instruct_contraire;
        break;
}
```

Avec la structure switch, on peut écrire l'exemple précédent comme ceci :

```
<script LANGUAGE='Javascript'>
    function VoirJourPair()
    {
        var jour = parseInt(document.getElementById('jour').value);
        switch(jour)
        {
            case 2:
                alert("Mardi");
                break;
            case 4:
                alert("Jeudi");
                break;
            case 6:
                alert("Samedi");
                break;
            default:
                alert("Désolé! On ne travaille que les jours pairs");
                break;
        }
    }
</script>
```

On peut également utiliser plus d'une condition avant d'exécuter une série d'instructions.

Exemple :

```

<html>
  <head>
    <title>JavaScript et HTML</title>
    <meta charset="UTF-8">
    <script LANGUAGE='Javascript'>
      function TestSexe()
      {
        var sexe = document.getElementById('sexe').value;
        switch(sexe)
        {
          case 'M': case 'm': alert("Vous êtes un homme"); break;
          case 'F': case 'f': alert("Vous êtes une femme"); break;
          default: alert("Désolé! Le sexe saisi est invalide"); break;
        }
      }
    </script>
  </head>
  <body>
    <form>
      <input type="text" id="sexe" placeholder="Votre sexe SVP"/>
      <input type="button" value="Test" onclick="TestSexe() ()"/>
    </form>
  </body>
</html>

```

II.7.2. Les structures itératives (boucles)

Les boucles permettent d'effectuer de manière répétitive une série d'opérations.

a. La boucle **while**

La boucle while permet d'exécuter les instructions tant que la condition est vraie.

Syntaxe :

```

while(condition_Vraie)
{
    instructions;
}

```

Si la condition est fausse au début, aucune instruction n'est exécutée.

Exemple : la somme de i allant de 1 à 10 :

```
<html>
  <head>
    <title>JavaScript</title>
    <meta charset="UTF-8">
    <script>
      function somme()
      {
        var i = 0;
        var som = 0;
        while (i <= 10)
        {
          som += i;
          i++;
        }
        alert("La somme de i allant de 1 à 10 vaut : " + som);
      }
    </script>
  </head>
  <body onload="somme()">
  </body>
</html>
```

b. La boucle do...while

Cette boucle fait la même chose que la boucle while. La seule différence est même si la condition est fausse au début, les instructions seront quand même exécutées une seule fois.

Syntaxe :

```
do
{
    instructions;
}while(condition_Vraie)
```

L'exemple peut s'écrire en utilisant la boucle do...while comme ceci :


```

<script>
    function somme()
    {
        var i = 0;
        var som = 0;
        do
        {
            som += i;
            i++;
        }while(i <= 10);
        alert("La somme de i allant de 1 à 10 vaut : " + som);
    }
</script>

```

c. La boucle for

Cette structure est utilisée si le nombre d'itération est connu d'avance. Elle contient un indice de la boucle qui est une valeur entière.

Syntaxe :

```

for (expression_du_depart; condition; incrementation) {
    instruction;
}

```

Notre exemple peut alors être écrit en utilisant la boucle for de la manière suivante :

```

<script>
    function somme()
    {
        var i;
        var som = 0;
        for(i=0; i<=10; i++)
        {
            som += i;
        }
        alert("La somme de i allant de 1 à 10 vaut : " + som);
    }
</script>

```

Remarquez qu'un même exemple est écrit de trois manières différentes en utilisant le trois sortes de boucles.

Remarquez aussi qu'avec les deux premières boucles, l'indice d'itération est initialisé au départ par une valeur entière avant d'être utilisé dans la boucle, contraire à la dernière où l'indice est initialisé dans la formulation de la boucle.

II.8. L'instruction **break**

L'instruction break permet d'interrompre prématurément une boucle for ou while.

Exemple : Affichage de chiffre allant de 1 à 10 :

- Sans utilisation de l'instruction break :

```
function sansBreak()  
{  
    chiffre = 1;  
    document.write("Affichage de 1 à 10: <br>");  
    while(chiffre < 11)  
    {  
        document.write( chiffre + "<br>");  
        chiffre++;  
    }  
}
```

Affichage de 1 à 10:
1
2
3
4
5
6
7
8
9
10

- Avec l'utilisation de l'instruction break

```
function avecBreak()
{
    chiffre = 1;
    document.write("Affichage de 1 à 10: <br>");
    while(chiffre < 11)
    {
        if(chiffre==5) break;
        document.write( chiffre + "<br>");
        chiffre++;
    }
}
```

Affichage de 1 à 10:

1
2
3
4

II.9. L'instruction continue

L'instruction continue permet de sauter une instruction dans une boucle for ou while et de continuer ensuite le bouclage (sans sortir de celui-ci comme le fait break).

Exemple :

```
function leContinue()
{
    chiffre = 1;
    document.write("Affichage de 1 à 10: <br>");
    while (chiffre < 11)
    {
        if (chiffre == 5)
        {
            chiffre++;
            continue;
        }
        document.write(chiffre + "<br>");
        chiffre++;
    }
}
```

Affichage de 1 à 10:

1
2
3
4
6
7
8
9
10

Remarquez bien que dans l'affichage on ne voit pas le chiffre 5 parmi les chiffres affichés.

II.10. JavaScript et le formulaire

II.10.1. Généralités

Bien que nous ayons essayé d'anticiper un peu plus, avec JavaScript, les formulaires HTML prennent une toute autre dimension. En JavaScript, il est possible d'accéder à chaque élément d'un formulaire pour, par exemple, y aller lire son contenu ou écrire une valeur, noter un choix auquel on pourra associer un gestionnaire d'événement. Tous ces éléments renforcent grandement les capacités interactives des pages web.

Mettons au point le vocabulaire que nous utiliserons. Un formulaire est l'élément Html déclaré par les balises `<form>` `</form>` (vu précédemment), contenant un ou plusieurs éléments que nous appellerons des contrôles (widget). Ces contrôles sont notés par exemple par la balise `<input type...>`.

II.10.2. Déclaration d'un formulaire

Ceci étant détaillé plus haut, il faudra noter qu'en JavaScript, pour désigner le chemin complet des éléments d'un formulaire on devra alors ajouter un autre attribut (**name**) au formulaire qui aura comme valeur le nom du formulaire. Sachant que, les attributs **action** et **method** sont facultatifs pour autant que jusqu'ici on ne fait pas appel au serveur.

Une erreur classique en JavaScript est, emporté par son élan, d'oublier de déclarer la fin du formulaire : `</form>` après avoir incorporé un contrôle.

II.10.3. Le contrôle ligne de texte

La zone de texte est l'élément d'entrée/sortie par excellence de JavaScript. La syntaxe Html est `<INPUT TYPE="text" NAME="nom" SIZE=x MAXLENGTH=y>` pour un champ de saisie d'une seule ligne, de longueur x et de longueur maximale de y.

L'objet text possède trois propriétés :

Propriété	Description
-----------	-------------

Name	indique le nom du contrôle par lequel on pourra accéder.
Default value	indique la valeur par défaut qui sera affichée dans la zone de texte.
Value	indique la valeur en cours de la zone de texte. Soit celle tapée par l'utilisateur ou si celui-ci n'a rien tapé, la valeur par défaut.

1. Lire une valeur dans une zone de texte

Voici un exemple que nous détaillerons :

Voici une zone de texte. Entrez une valeur et appuyer sur le bouton pour contrôler celle-ci.



Le code associé est :

```
<html>
  <head>
    <title>JV et Formulaire</title>
    <meta charset="UTF-8">
    <script>
      function controleText(form1)
      {
        var text= document.form1.entree.value;
        alert("Vous avez saisi : "+text);
      }
    </script>
  </head>
  <body>
    <form name="form1">
      <input type="text" name="entree" value="">
      <input type="button" name="bouton" value="Contrôler" onclick="controleText(form1)"/>
    </form>
  </body>
</html>
```

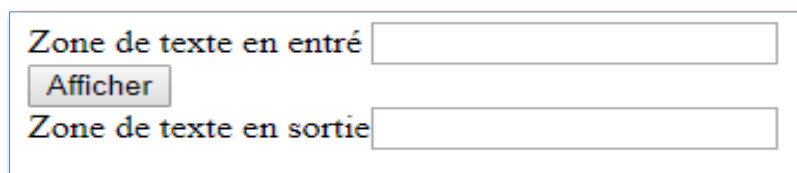
Lorsqu'on clique le bouton "contrôler", JavaScript appelle la fonction **controleText()** à laquelle on passe le formulaire dont le nom est form1 comme argument.

Cette fonction **controleText()** définie dans les balises <head> prend sous la variable test, la valeur de la zone de texte. Pour accéder à cette valeur, on note le chemin complet de celle-ci. Soit dans le document présent, il y a l'objet formulaire appelé **form1** qui contient le contrôle de texte nommé **entree** et qui a comme propriété l'élément de valeur value.

Ce qui donne **document.form1.entree.value**.

2. Ecrire une valeur dans une zone de texte

Entrez une valeur quelconque dans la zone de texte d'entrée. Appuyer sur le bouton pour afficher cette valeur dans la zone de texte de sortie.



The screenshot shows a web form with a light blue border. It contains two text input fields. The top field is labeled "Zone de texte en entré" and is empty. Below it is a button labeled "Afficher". Below the button is another text input field labeled "Zone de texte en sortie", which is also empty.

Code associé :

```
<html>
  <head>
    <title>JV et Formulaire</title>
    <meta charset="UTF-8">
    <script>
      function afficherText(form2)
      {
        var lire= document.form2.entree.value;
        document.form2.sortie.value=lire;
      }
    </script>
  </head>
  <body>
    <form name="form2">
      <label>Zone de texte en entré</label> <input type="text" name="entree" value="" /><br/>
      <input type="button" name="bouton" value="Afficher" onclick="afficherText(form2)" /><br/>
      <label>Zone de texte en sortie</label><input type="text" name="sortie" value="" />
    </form>
  </body>
</html>
```

Lorsqu'on clique le bouton "Afficher", JavaScript appelle la fonction **afficherText()** à laquelle on passe le formulaire dont le nom est cette fois form2 comme argument.

Cette fonction **afficherText()** définie dans les balises <head> prend sous la variable **lire**, la valeur de la zone de texte d'entrée. A l'instruction suivante, on dit à JavaScript que la valeur de la zone de texte **sortie** comprise dans le formulaire nommé form2 est celle de la variable **lire**. A nouveau, on a utilisé le chemin complet pour arriver à la propriété valeur de l'objet souhaité soit en JavaScript

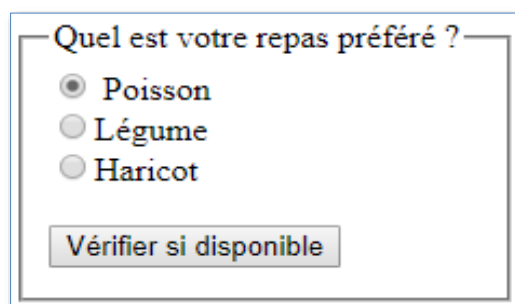
document.form2.sortie.value.

3. Les boutons radio

Les boutons radio sont utilisés pour noter un choix, et seulement un seul, parmi un ensemble de propositions.

Propriété	Description
Name	indique le nom du contrôle. Tous les boutons portent le même nom
Checked	Indique l'état en cours de l'élément radio
Defaultchecked	indique l'état du bouton sélectionné par défaut.
Value	indique la valeur de l'élément radio.

Exemple : choix du repas préféré :



Quel est votre repas préféré ?

☒ Poisson

☐ Légume

☐ Haricot

Vérifier si disponible

Code associé :

```

<html>
  <head>
    <title>JV et Formulaire</title>
    <meta charset="UTF-8">
    <script language="JavaScript">
      function choixRepas(form3)
      {
        if (form3.repas[0].checked)
          alert("Donc on vous prépare alors du " + form3.repas[0].value);

        if (form3.repas[1].checked)
          alert("Donc on vous prépare alors de " + form3.repas[1].value);

        if (form3.repas[2].checked)
          alert("Donc on vous prépare alors le " + form3.repas[2].value);
      }
    </script>
  </head>
  <body>
    <form name="form3">
      <fieldset style="width: 200px; height: 120px;">
        <legend>Quel est votre repas préféré ?</legend>
        <input type="radio" name="repas" value="Poisson" id="choix1" />
        <label for="choix1">Poisson</label><br />
        <input type="radio" name="repas" value="Legume" id="choix2">
        <label for="choix2">Légume</label><br />
        <input type="radio" name="repas" value="Haricat" id="choix3">
        <label for="choix3">Haricot</label><br /><br />
        <input type="button" name="but" value="Vérifier si disponible"
          onClick="choixRepas(form3)" />
      </fieldset>
    </form>
  </body>
</html>

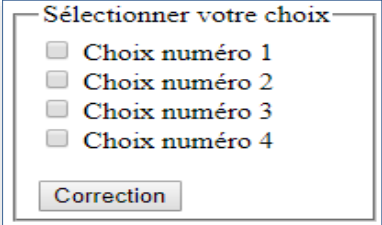
```

4. Les boutons case à cocher (checkbox)

Les boutons case à cocher sont utilisés pour noter un ou plusieurs choix (pour rappel avec les boutons radio un seul choix) parmi un ensemble de propositions. A part cela, sa syntaxe et son usage est tout à fait semblable aux boutons radio sauf en ce qui concerne l'attribut **name**.

Propriété	Description
Name	indique le nom du contrôle. Toutes les cases à cocher portent un nom différent
Checked	Indique l'état en cours de l'élément case à cocher
Defaultchecked	indique l'état du bouton sélectionné par défaut.
Value	indique la valeur de l'élément case à cocher.

Exemple :



Dans le formulaire nommé form4, on déclare quatre cases à cocher. Notez que l'on utilise un nom différent pour les quatre boutons. Vient ensuite un bouton qui déclenche la fonction réponse ().

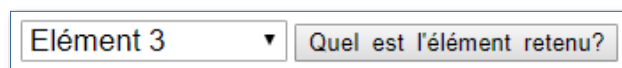
Cette fonction teste quelles cases à cocher sont sélectionnées. Pour avoir la bonne réponse, il faut que les cases 1, 2 et 4 soient cochées. On accède aux cases en utilisant chaque fois leur nom. On teste la propriété checked du bouton par (form4.nom_de_la_case.checked). Dans l'affirmative (&& pour et logique), une boîte d'alerte s'affiche pour la bonne réponse. Dans la négative, une autre boîte d'alerte vous invite à recommencer.

5. Liste de sélections (déroulante)

Le contrôle liste de sélection vous permet de proposer diverses options sous la forme d'une liste déroulante dans laquelle l'utilisateur peut cliquer pour faire son choix. Ce choix reste alors affiché. La boîte de la liste est créée par la balise <select> et les éléments de la liste par un ou plusieurs tags <option>. La balise </select> termine la liste.

Propriété	Description
Name	indique le nom de la liste déroulante
Length	Indique le nombre d'éléments de la liste. S'il est indiqué dans le tag <select>, tous les éléments de la liste seront affichés. S'il n'est pas indiqué, un seul apparaîtra dans la boîte de la liste déroulante.
selectedIndex	Indique le rang à partir de 0 de l'élément qui a été sélectionné par l'utilisateur
Defaultselected	indique l'élément de la liste sélectionné par défaut. C'est lui qui apparaît alors dans la petite boîte.

Exemple :



The image shows a web form with two elements. On the left is a dropdown menu with a small downward arrow on the right, and the text 'Elément 3' is displayed. To the right of the dropdown is a text input field with a light gray border and a small shadow, containing the text 'Quel est l'élément retenu?'.

Code associé :

```

<html>
  <head>
    <title>JS et Formulaire</title>
    <meta charset="UTF-8">
    <script language="javascript">
      function liste(form5)
      {
        alert("L'\élément " + (form5.list.selectedIndex + 1));
      }
    </SCRIPT>
  </head>
  <body>
    <form name="form5">
      <select name="list" style="width: 150px; font-size: 17px;">
        <option value="1">Elément 1
        <option value="2">Elément 2
        <option value="3">Elément 3
      </select>
      <input type="button" name="btn" value="Quel est l'élément retenu?"
        onClick="liste(form5)"/>
    </form>
  </body>
</html>

```

Dans le formulaire nommé form5, on déclare une liste de sélection par la balise `<SELECT>` `</SELECT>`. Entre ses deux balises, on déclare les différents éléments de la liste par autant de tags `<OPTION>`. Vient ensuite un bouton qui déclenche la fonction `liste ()`. Cette fonction teste quelle option a été sélectionnée. Le chemin complet de l'élément sélectionné est `form5.nomdelaliste.selectedIndex`.

Comme l'index commence à 0, il ne faut pas oublier d'ajouter 1 pour retrouver le juste rang de l'élément.

6. Le contrôle textarea (pour les bavards).

L'objet `textarea` est une zone de texte de plusieurs lignes.

La syntaxe Html est :

```
<form>
  <textarea name="nom" rows="x" cols="y">
    Texte par défaut
  </textarea>
</form>
```

Où rows="x" représente le nombre de lignes et cols="y" représente le nombre de colonnes.

L'objet textarea possède plusieurs propriétés :

Propriété	Description
Name	indique le nom du contrôle par lequel on pourra accéder
DefaultValue	indique la valeur par défaut qui sera affichée dans la zone de texte.
Value	indique la valeur en cours de la zone de texte. Soit celle tapée par l'utilisateur ou si celui-ci n'a rien tapé, la valeur par défaut.

A ces propriétés, il faut ajouter onFocus(), onBlur(), onSelect() et onChange().

En JavaScript, on utilisera \r\n pour passer à la ligne.

Comme par exemple dans l'expression document.Form.Text.value = 'Check\r\nthis\r\nout'.

7. Le contrôle Reset.

Le contrôle Reset permet d'annuler les modifications apportées aux contrôles d'un formulaire et de restaurer les valeurs par défaut.

La syntaxe Html est : `<input type="reset" name="nom" value="texte" />`

Où **value** donne le texte du bouton.

Une seule méthode est associée au contrôle Reset, c'est la méthode **onClick()**. Ce qui peut servir, par exemple, pour faire afficher une autre valeur que celle par défaut.

8. Le contrôle Submit

Le contrôle à la tâche spécifique de transmettre toutes les informations contenues dans le formulaire à l'URL désignée dans l'attribut ACTION du tag <FORM>.

La syntaxe Html est :

```
<input type="submit" name="nom" value="texte" />
```

Où **value** donne le texte du bouton.

Une seule méthode est associée au contrôle Submit, c'est la méthode **onClick()**

9. Le contrôle Hidden (caché)

Le contrôle Hidden permet d'entrer dans le script des éléments (généralement des données) qui n'apparaîtront pas à l'écran. Ces éléments sont donc cachés. D'où son nom.

La syntaxe Html est :

```
<input type="hidden" name="nom" value="les données cachées">
```

10. L'envoi d'un formulaire par email.

Uniquement Netscape !!!

A force de jouer avec des formulaires, il peut vous prendre l'envie de garder cette source d'information. Mais comment faire? JavaScript, et à fortiori le Html, ne permet pas d'écrire dans un fichier. Ensuite, le contrôle Submit est surtout destiné à des CGI ce qui entraîne (encore) un codage spécial à maîtriser. D'autant que pour nous simples et présumés incompetents internautes, la plupart des providers ne permettra pas d'héberger une CGI faite par un amateur pour des raisons (tout à fait compréhensibles) de sécurité. Il ne reste plus que l'unique solution de l'envoi d'un formulaire via le courrier électronique.

Exemple :

Code associé :

```
<html>
  <head>
    <title>Envoi du formulaire</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <fieldset style="width: 250px; height: 150px;">
      <legend>Envoi du formulaire par email</legend>
      <form method="post" action="mailto:votre_adresse_Email">
        <input type="text" name="nom" placeholder="Votre nom"
          style="width: 100%; font-size: 17px;"><br/><br/>
        <textarea name="adresse" rows=3 cols=35 placeholder="Votre adresse"
          style="width: 100%;">
        </textarea><br/><br/>
        <input type="submit" value="Soumettre" style="width: 50%; font-size: 15px;">
      </form>
    </fieldset>
  </body>
</html>
```

Attention ! Ceci ne marche que sous Netscape et pas sous Microsoft Explorer 3.0 ... Avec Explorer, le mailto ouvre le programme de Mail mais n'envoie rien du tout.

CHAPITRE III : LANGAGE DE PROGRAMMATION DU COTE SERVEUR : PHP

III.1. Introduction

PHP (HyperText Préprocesseur) est un langage de script qui est principalement utilisé pour être exécuté par un serveur http. Il est beaucoup plus utilisé pour créer des sites web dynamiques. Une page HTML qui est directement décodée par un navigateur (Internet Explorer, Google Chrome,... par exemple). Par contre, le PHP est décodé par le serveur qui transfère le résultat vers les navigateurs au format HTML. Pour l'utilisateur cette fonctionnalité est complètement transparente.

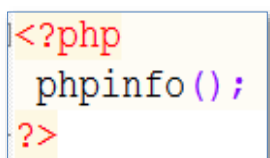
Le PHP est généralement installé sur un serveur Web et couplé à une base de données MySQL. Ce n'est pas obligatoire, mais permet des applications étendues comme la création de forum, site de vente en ligne...

On désigne parfois PHP comme une plate-forme plus qu'un simple langage.

Sa syntaxe et sa construction ressemblent à celles des langages C++ et Perl, à la différence que le PHP peut être directement intégré dans du code HTML.

Créer des pages PHP n'oblige pas à créer tout le site dans ce langage. Les pages peuvent être mélangées avec d'autres sur le même site. La seule manière de vérifier le type de page est de vérifier l'extension de la page internet, et encore puisse que des pages en HTML peuvent être simplement renommées en PHP pour des développements futurs.

Pour vérifier quelle version utilise votre hébergeur, tapez simplement les lignes suivantes dans un éditeur puis exécuter les :



```
<?php
phpinfo();
?>
```

Nous venons finalement de programmer notre première commande en PHP. Analysons les lignes de ce fichier :

<php> : Cette ligne explique que nous avons démarré une liste de commande PHP.

phpinfo() ;: est une commande du langage de programmation PHP qui affiche la configuration du serveur

?> : Ici termine une liste d'instruction PHP.

On peut alors utiliser plusieurs séries d'instructions (chaque fois avec les mêmes délimiteurs) dans une page, entrecoupée des parties en html. L'extension du fichier sera alors obligatoirement **.php**.

III.2. Fonctionnalités du langage

Pour débiter, nous allons nous inspirer d'un programme en HTML pour le modifier en PHP. Nous en profiterons pour voir quelques instructions simples comme l'affichage de texte et les formats de dates.

```
<html>
  <head>
    <title>PHP</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <h3>Ma première instruction</h3>
    <p>Bienvenu dans la programmation Web</p>
  </body>
</html>
```

Ce fichier html est relativement simple, il ne fait que mettre un titre dans l'entête et affiche un texte dans un paragraphe. Modifions la programmation pour le créer en PHP. Remplaçons simplement le **<p>** par :

```
<?php echo "Bienvenu dans la programmation Web"; ?>
```


L'instruction **print()** est insérée entre deux balises qui délimite le PHP. Elle permet d'afficher un texte sur l'écran. Les autres parties en HTML ne sont pas modifiées.

Noter que chaque ligne d'instruction doit terminer par un point-virgule (;). Sauf quoi, vous aurez une erreur de type :

A screenshot of a PHP error message in a browser. It shows a red exclamation mark icon followed by the text: "Parse error: syntax error, unexpected 'echo' (T_ECHO), expecting ',' or ';' in C:\wamp64\www\FirstAppPHP\index.php on line 10".

Parse error: syntax error, unexpected 'echo' (T_ECHO), expecting ',' or ';' in C:\wamp64\www\FirstAppPHP\index.php on line 10

Ceci arrive souvent quand on ne met le point-virgule à la fin d'une instruction avant de passer à une autre.

Nous aurions pu utiliser l'instruction **print()** similaire à **echo**. Les lignes d'instructions précédentes deviennent alors :

```
<?php
print "Bienvenu dans la programmation Web";
?>
```

Pour afficher du texte, nous utilisons les guillemets (simples ' ou double " au choix). Mais pour insérer d'autres caractères spéciaux comme par exemple l'apostrophe ('), il faut alors précéder ce dernier par le caractère anti slash (\).

Par exemple :

```
<?php
print 'J\'ai réussi ma première instruction en PHP';
?>
```

Les caractères de contrôle html
, <hr />...

L'instruction `echo '<br />';` permet de faire un saut de ligne.

Les commentaires en PHP peuvent se présenter sous deux formes : /* */ ou //

III.3. Les variables

Le langage PHP permet de manipuler des variables et constantes. Voici les types les plus manipulés en PHP :

1. **booléen** : true ou false (vraie ou faux) ;
2. **entier** : acceptant un nombre entier (sans virgule)
3. **chiffre à virgule flottante** : (c'est automatiquement un nombre décimale de type double), nombre avec chiffre derrière la virgule comme par exemple : 0.82, 11.407. remarquez le point comme séparateur. (considéré ici comme virgule)
4. **chaîne de caractères** : du texte encadré par " ou '. Exemple "Ceci est un texte".
5. **Tableau** : exemple **array(2,3)** ;
6. **Objets** : (images, lien texte...).

A noter que le PHP ne demande pas de spécifications de type de variable préalable, ni même de les déclarer (sauf les tableaux). Ceci dit : PHP n'est pas un langage typé.

III.3.1. Déclaration d'une variable

Quelques consignes pour la déclaration des variables :

a. Les noms de variables doivent :

- commencer par le symbole dollar (\$) ;
- inclure des lettres, des chiffres et le caractère (_) commencer par une lettre ou (_) après le symbole (\$).

b. Les noms de variables ne doivent pas :

- inclure les caractères réservés : - @ , . ; : / < \ >
- inclure des espaces.

En plus de tout ça, le nom de variable doit quand même significatif. Il peut inclure des caractères accentués mais ce n'est pas souhaitable. Le PHP est un langage est sensible à la casse (c'est-à-dire, il sait faire la différence entre le MAJUSCULE et le minuscule).

Pour déclarer une variable rien de plus simple :

```
$nom_variable;
```

Exemple :

```
$nom;
```

Mais quelque problème se pose si on veut vraiment si la variable existe. Essayons d'anticiper quelque chose du genre méthode ici (**isset**) qui permet de vérifier l'existence d'une variable en PHP.

La variable ainsi déclarée, si on vérifie son existence de la manière :

```
<?php
$nom;
if(isset($nom))
    print "La variable existe";
else
    print "La variable n'existe pas";
?>
```

Ce qui donne comme résultat dans le navigateur :

La variable n'existe pas

Pourtant, la variable **\$nom** a été bien déclarée au départ.

Tout ça, parce que bien qu'elle a été déclarée, mais elle n'a pas été encore initialisée, le serveur ne reconnaît pas encore. Voilà le pourquoi de son inexistence. Si on tente de faire comme ceci :

```
<?php
$nom="";
if(isset($nom))
    print "La variable existe";
else
    print "La variable n'existe pas";
?>
```

Cette fois ci, le résultat est bien celui entendu :

La variable existe

Donc, il est surtout recommandé d'initialiser la variable pendant sa déclaration. Mais surtout encore par une valeur différente de nulle (**null**). C'est très important.

Exemple :

```
<?php
$nom="Kanyinda Kam's";
print $nom;
?>
```

Résultat dans le navigateur :

Kanyinda Kam's

On peut afficher la variable (son contenu) de nouveau entre les guillemets comme une chaîne de caractères si on veut la concaténer avec une chaîne.

Exemple :

```
<?php
$nom="Kanyinda Kam's";
print "Bonjour $nom";
?>
```

Ce qui donne ça dans le navigateur :

Bonjour Kanyinda Kam's

Mais il y a un symbole spécial pour la concaténation. C'est le signe point (.).

D'où l'exemple précédent peut s'écrire comme ceci :

```
<?php
$nom="Kanyinda Kam's";
print "Bonjour " . $nom;
?>
```

Le résultat sera toujours le même.

III.4. Les constantes

Une constante est un identifiant (un nom) qui représente une valeur simple. Comme son nom le suggère, cette valeur ne peut jamais être modifiée durant l'exécution du script (les constantes magiques **__FILE__** et **__LINE__**

sont les seules exceptions). Le nom d'une constante est sensible à la casse, par défaut. Par convention, les constantes sont toujours en majuscules.

Les noms de constantes suivent les mêmes règles que n'importe quel nom en PHP.

Tous comme les superglobals, les constantes sont accessibles de manière globale. Vous pouvez la définir n'importe où, et y accéder depuis n'importe quelle fonction.

Avec PHP, les constantes sont définies grâce à la fonction **define()**.

La syntaxe de la fonction **define ()** est la suivante :

```
define ("NOM_DE_LA_CONSTANTE", Valeur);
```

Exemple :

```
define("TVA",16.4);
```

Seuls les types de données scalaires peuvent être placés dans une constante : c'est à dire les types booléens, entier, double et chaîne de caractères (soit bool, integer, double et string).

Pour utiliser ou afficher une constante, on indique seulement son nom dans l'expression :

```
<?php  
define("TVA", 16.4);  
echo 'La valeur de la TVA est ' . TVA;  
?>
```

Dans le navigateur, on a comme résultat :

```
La valeur de la TVA est 16.4
```

III.5. Les opérateurs

III.5.1. Opérateurs Arithmétiques

Les opérations arithmétiques sont : l'addition, la multiplication, la soustraction et la division (entière et réelle).

Opérateur	Symbole utilisé
Addition	+
Soustraction	-
Multiplication	*
Division	/
Modulo	%

III.5.2. Opérateurs Logiques

Opérateur	Symbole utilisé
NON	!
ET	&& ou and
OU inclusif	ou or
OU exclusif	Xor

III.5.3. Opérateurs Relationnels

Les opérateurs relationnels permettent d'effectuer des comparaisons dans des conditionnelles par exemple, le résultat ainsi retourné est un booléen.

Opérateur	Symbole utilisé
Strictement inférieur à	<
Strictement supérieur	>
Inférieur ou égal à	<=
Supérieur ou égal à	>=
Égal à	==
Différent de	!=

III.5.4. Opérateurs d'incrémentation et décrémentation.

A. L'incrémentation.

L'incrémentation est équivalente à ces expressions : $\$i = \$i + 1$ ou encore $\$i += 1$

L'expression suivante : $++\$i$, a pour effet d'incrémenter de 1 la valeur de i , et sa valeur de est celle de i après incrémentation.

En revanche, lorsque cet opérateur est placé après son unique opérande, la valeur de l'expression correspondante est celle de la variable avant incrémentation.

On dit que $++$: est un opérateur de pré incrémentation lorsqu'il est placé à gauche de son opérande un opérateur de post incrémentation lorsqu'il est placé à droite de son opérande

B. La décrémentation.

La décrémentation est équivalente à ces expressions : $\$i = \$i - 1$ ou encore $\$i -= 1$

L'expression suivante : $--\$i$, a pour effet de décrémentation de 1 la valeur de i , et sa valeur de est celle de i après décrémentation.

En revanche, lorsque cet opérateur est placé après son unique opérande, la valeur de l'expression correspondante est celle de la variable avant décrémentation.

On dit que $--$ est :

- un opérateur de pré décrémentation lorsqu'il est placé à gauche de son opérande
- un opérateur de post décrémentation lorsqu'il est placé à droite de son opérande.

III.1.5.5. Opérateurs d'affectation élargie

Lorsqu'on veut effectuer une opération dont le résultat dépend de cette même variable, il est plus simple de recourir aux opérateurs d'affectation élargie.

Exemple : Au lieu d'écrire ceci : `$a = $a + $b;`

On peut directement procéder ainsi : `$a += $b;`

Ce genre d'opérateur est valable pour tous les opérateurs arithmétiques ainsi que pour l'opérateur de concaténation, par exemple : `$a.= " et bienvenue";` équivaut à `$a = $a. " et bienvenue";`

III.6. Affichage du texte

Bien que nous ayons l'anticiper plus haut, PHP fournit 3 fonctions permettant d'envoyer du texte au navigateur. Ces fonctions ont la particularité de pouvoir insérer dans les données envoyées des valeurs variables, pouvant être fonction d'une valeur récupérée par exemple, c'est ce qui rend possible la création de pages dynamiques.

Les 3 fonctions sont les suivantes :

- `echo`
- `print`
- `printf`

III.6.1. Echo

La fonction `echo` permet d'envoyer au navigateur la chaîne de caractères (délimitée par des guillemets) qui la suit comme nous l'avons vu et manipuler plusieurs fois plus haut.

La syntaxe de cette fonction est la suivante :

echo Expression;

L'expression peut être une chaîne de caractères ou une expression que l'interpréteur évalue

```
echo (15+2)-4;
```


III.6.2. Print

La fonction `print` est similaire à la fonction `echo`. La syntaxe de la fonction `print` est la suivante :

`print (expression);`

L'expression peut, comme pour la fonction `echo` être une chaîne de caractères ou une expression que l'interpréteur évalue :

```
<?php
print 'Bonjour les amis';
print 15 + 2 - 4;
?>
```

Noter qu'avec **`echo`** et **`print`** on peut placer les expressions à afficher entre les parenthèses comme vu dans les exemples plus haut.

Exemple :

```
<?php
echo ('Bonjour les amis');
print (15 + 2 - 4);
?>
```

III.6.3. Printf

La fonction **`printf ()`** (empruntée au langage C) est rarement utilisée car sa syntaxe est plus lourde. Toutefois, contrairement aux deux fonctions précédentes, elle permet un formatage des données, cela signifie que l'on peut choisir le format dans lequel une variable sera affichée à l'écran.

La syntaxe de **`printf ()`** est la suivante : `printf (chaîne formatée);`

Une chaîne formatée est une chaîne contenant des codes spéciaux permettant de repérer l'emplacement d'une valeur à insérer et son format, c'est-à-dire sa représentation. A chaque code rencontré doit être

associé une valeur ou une variable, que l'on retrouve en paramètre à la fin de la fonction printf.

III.7. Les structures de contrôles

III.7.1. Les structures alternatives

a. si simple (if)

Cette forme de **if** simple permet l'exécution d'une série d'instructions que si la condition est satisfaisante. Au cas contraire rien ne va se produire.

Syntaxe :

```
if (condition1) {  
    instructions;  
}
```

Exemple :

```
<?php  
$sexe='M';  
if ($sexe=="M") {  
    print "Bonjour Monsieur";  
}  
?>
```

b. if...else

Cette forme a généralement deux blocs des séries d'instruction, le premier est exécuté si la condition est satisfaisante au cas contraire le second.

Syntaxe :

```
if (condition) {  
    instructions1;  
}else{  
    instructions2;  
}
```

Exemple :

```
<?php
$sexe='F';
if ($sexe=="M") {
    print "Bonjour Monsieur";
}else{
    echo 'Bonjour Madame';
}
?>
```

c. if...else if...else

Cette forme permet d'exécuter plus d'un bloc d'instructions chacun selon la condition associée.

Syntaxe :

```
if (condition1) {
    instructions1;
}else if(condition2)
{
    instruction2;
}else if(conditionN){
    instructionN;
}else{
    instruction_autre;
}
```

Exemple :

```
<?php
$sexe='F';
if ($sexe=="M") {
    print "Bonjour Monsieur";
}else if($sexe=="F"){
    echo 'Bonjour Madame';
}else{
    echo "Sexe incorrect";
}
?>
```

La structure **else if** pouvant se répéter autant de fois que des conditions prévues l'exigeront.

Noter que, si pour une condition il y a qu'une seule instruction à exécuter les accolades ne sont pas nécessaires.

L'exemple précédent peut s'écrire autrement de la manière suivante :

```
<?php
$sexe='M';
if ($sexe=="M")
    print "Bonjour Monsieur";
else if($sexe=="F")
    echo 'Bonjour Madame';
else
    echo "Sexe incorrect";
?>
```

On utilise les opérateurs logiques quand il s'agit de vérifier plus d'une condition au même moment avant d'exécuter une série d'instruction.

Exemple :

```
<?php
$sexe = 'F';
$age = 19;
if ($sexe == "M" and $age > 18)
    print "Bonjour Monsieur";
else if ($sexe == "F" || $age < 18)
    echo 'Bonjour Madame';
else
    echo "Les données ne sont pas valide";
?>
```

Résultat : Bonjour Monsieur

d. La structure switch

La structure switch permet d'éviter la lourdeur de l'écriture de la structure **if...elseif...else**. Elle permet en outre une meilleure lisibilité.

La structure switch a comme syntaxe :

```

switch ($variable)
{
    case valeur1:
        instruction1;
        break;
    case valeur2:
        instruction2;
        break;

    case valeurN:
        instructionsN;
        break;
    default :
        instruction_par_defaut;
        break;
}

```

III.7.2. Les structures itératives (boucle)

a. Boucle while

Syntaxe :

```

<?php
while (condition_Vraie) {
    instruction;
}
?>

```

Exemple : la table de multiplication par 12

```

<?php
$i=1;
echo "LA TABLE DE MULTIPLICATION PAR 6<br/>";
while ($i<13) {
    echo " $i x 6 = " . ($i*6) . "<br/>";
    $i++;
}
?>

```

Résultat : On aura la table de multiplication par 6

b. Boucle do...while

La boucle do...while fait la même chose que while, à la seule différence que cette dernière exécute au moins une fois l'instruction avant de vérifier la condition. Syntaxe :

```
do{  
    instructions;  
}while(condition_Vaire);
```

L'exemple précédent peut être écrit en utilisant la boucle do...while comme ceci :

```
<?php  
$i=1;  
echo"LA TABLE DE MULTIPLICATION PAR 6<br/>";  
do{  
    echo"    $i x 6 = ".$i*6."<br/>";  
    $i++;  
}while($i<13);  
?>
```

Le résultat est le même avec l'utilisation de la boucle while.

Remarque : Quand on utilise les boucles while et do...while la variable d'incrément doit être initialisée au départ.

c. Boucle for

Syntaxe :

```
for(initialisation; condition; incrémentation)  
{  
    instructions;  
}
```

L'exemple précédent peut encore s'écrire en utilisant la boucle **for** comme ceci :

```
<?php
echo "LA TABLE DE MULTIPLICATION PAR 6<br/>";
for ($i = 1; $i <= 12; $i++) {
    echo "    $i x 6 = " . ($i * 6) . "<br/>";
}
?>
```

Résultat : On aura la table de multiplication par 6

III.8. Les formats de date

Il existe plusieurs opérateurs pour la fonction **date()**. Mais nous n'allons pas utiliser tous ici.

Format	Description
J	Jour du mois, sans les 0, soit de 1 à 31
D	Jour du mois avec 2 chiffres, soit de 01 à 31
D	Jour de la semaine sur 3 lettres, en anglais
L	Jour de la semaine (en anglais)
M	Mois, de 01 à 12
N	Mois, sans les 0 initiaux (de 1 à 12)
M	trois premières lettres du mois en anglais
Y	année en 4 chiffres
Y	année en 2 chiffres
Z	jour de l'année (de 0 à 365)
H	heure, de 0 à 12
H	heure, de 0 à 24
S	secondes avec les 0, de 00 à 59
S	secondes, sans les zéros.
A	am pour le matin, pm pour l'après-midi
A	AM pour le matin, PM pour l'après-midi

Exemple :

```
<?php
echo "Nous sommes aujourd'hui le ".date('d/m/Y');
?>
```

Résultat :

III.9. Les fonctions

PHP propose une palette approximative de 2000 fonctions prédéfinies. Toutefois il vous est possible de créer vos propres bibliothèques de fonctions pour vos usages spécifiques, une meilleure lisibilité du code et une réutilisation.

1. Définition des fonctions

Les fonctions peuvent se distinguer en deux sous-groupes :

- Celles qui effectuent un traitement (affichage par exemple)
- Celles qui effectuent un traitement et retournent un résultat.

La syntaxe pour définir une fonction est la suivante :

```
<?php
function nom_de_la_fonction($arg1, $arg2,..., $argN)
{
    /traitement sur les paramètres effectués;
}
?>
```

L'appel de la fonction se fait de la manière suivante :

```
nom_de_la_fonction($argu);
```

Exemple :

```

<?php
function sommation($a, $b)
{
    $resultat=$a+$b;
    return $resultat;
}
$un=15; $deux=23;
echo "La somme de $un et de $deux vaut ". sommation($un, $deux);
?>

```

Dans le cas suivant un traitement est effectué à l'intérieur de la fonction puis retourné par cette dernière. De ce fait la fonction doit donc être affectée à une variable.

Exemple :

```

<?php

function sommation($a, $b) {
    $resultat = $a + $b;
    return $resultat;
}

$un = 15;
$deux = 23;
$somme = sommation($un, $deux);
echo "La somme de $un et de $deux vaut " . $somme;
?>

```

2. Librairies des fonctions

Idéalement toutes les fonctions créées devraient être regroupées dans un même fichier créant ainsi une bibliothèque de fonctions. Ce fichier sera appelé à l'intérieur des autres fichiers par le biais de la fonction **include**.

Exemple :

page1.php (fichier à inclure)

```

<?php
function sommation($a, $b)
{
    $resultat=$a+$b;
    return $resultat;
}
function multiplication($x, $y)
{
    return $x*$y;
}
function salutation($nom)
{
    echo "Bonjour ".$nom;
}
?>

```

page2.php

```

<?php
include 'page.php';
$un = 15;
$deux = 23;
$somme = sommation($un, $deux);
echo "La somme de $un et de $deux vaut " . $somme;
echo '<br/>';
echo "Le produit de $un et de $deux vaut " . multiplication($un, $deux);
echo '<br/>';
$nom="Asabi";
salutation($nom);
?>

```

III.10. Tableaux

1. Tableaux numérotés et tableaux associatifs

Il existe deux types de tableaux de variables sous PHP :

- Les tableaux à index numériques (tableaux numérotés) dans lesquels l'accès à la valeur de la variable passe par un index numérique .

Ex : \$tableau[0], \$tableau[1], \$tableau[2], ...

- Les tableaux à index associatifs (ou tableaux associatifs) dans lesquels l'accès à la valeur de la variable passe par un index nominatif.

Ex : \$tableau[nom], \$tableau[prénom], \$tableau[adresse], ...

```
<?php
    $tableau = array(val0, val1, val2,...);
    //Accès à chacune des valeurs
    $tableau[0] donnera val0;
    $tableau[1] donnera val1;
    $tableau[2] donnera val2;
    ...

    //Tableau à index associatif
    $tableau= array(variable1=>val1, variable2=>val2,...);

    $tableau[variable1]=donnera val1;
    $tableau[variable2]=donnera val2;
?>
```

Exemple :

```
<?php
//Tableau à index numéroté
$nom = array("Kanyinda", "Kayembe", "Kam's");
echo "<b>Le contenu du tableau numéroté</b><br/>";
echo $nom[0]."<br/>".$nom[1]."<br/>".$nom[2]."<br/>";

//Tableau à index associatif
$nom= array("nom"=>"Kanyinda", "postnom"=>"Kaymbe", "adresse"=>"Limete");
echo "<b>Le contenu du tableau associatif</b><br/>";
echo $nom["nom"]." - ".$nom["postnom"]." | ".$nom["adresse"];
?>
```

2. Parcours des tableaux.

PHP intègre une structure de langage qui permet de parcourir un à un les éléments d'un tableau :

foreach ().

Syntaxe :

```
foreach ($tableau as $valeur)
{
    //Appeller ici la valeur courante para $valeur
    ...
}
```

Exemple :

```
<?php

//Tableau à index numéroté
$nom = array("Kanyinda", "Kayembe", "Kam's");
echo "<b>Le contenu du tableau numéroté</b><br/>";
foreach ($nom as $el) {
    echo $el . " ";
}

$nom = array("nom" => "Kanyinda",
            "postnom" => "Kayembe",
            "adresse" => "Limete"
            );
echo "<br/><b>Le contenu du tableau associatif</b><br/>";
foreach ($nom as $val) {
    echo $val . " ";
}
?>
```

foreach() aussi bien énumérer le nom des variable que leur valeur.

Exemple :

```

$nom = array("nom" => "Kanyinda",
            "postnom" => "Kaymbe",
            "adresse" => "Limete"
            );
echo "<br/><b>Le contenu du tableau associatif</b><br/>";
foreach ($nom as $variable=>$valeur) {
    echo $valeur . " ";
}

```

3. Recherche dans un tableau.

- **array_key_exists ()** permet de vérifier si dans un tableau associatif une variable associative (clef) existe ou non. Elle retourne donc une valeur booléenne (True ou False).

Syntaxe :

```
$resultat= array_key_exists("nom_de_la_variable", $tableau);
```

Exemple :

```

<?php
//Recherche dans un tableau
$tableau = array("nom" => "Kanyinda", "postnom" => "Kaymbe", "adresse" => "Limete");

$recherche=array_key_exists("nom", $tableau);
if($recherche==true)
{
    echo "La variable <b>nom</b> se trouve dans le tableau<br/>";
    echo "et a pour valeur <b>".$tableau['nom']. "</b>";
}else{
    print("La variable <b>nom</b> n'existe pas");
}
?>

```

- **in_array ()** permet de déterminer si une valeur existe dans le tableau. Elle retourne donc une valeur booléenne (True ou False).

Syntaxe :

```
$resultat= in_array('nom_de_la_variable', tableau_associatif);
```

Exemple :

```
<?php
//Recherche dans un tableau
$tableau = array("nom" => "Kanyinda", "postnom" => "Kaymbe", "adresse" => "Limete");

$recherche= in_array("Kanyinda", $tableau);
if($recherche==true)
{
    echo "La valeur <b>Kanyinda</b> se trouve dans le tableau<br/>";
}
else{
    print("La variable <b>Kanyinda</b> n'existe pas dans le tableau");
}
?>
```

- **array_search** fonctionne comme in_array : il travaille sur les valeurs d'un tableau. Si il a trouvé la valeur, array_search renvoie la clé correspondante (c'est-à-dire le numéro si c'est un tableau numéroté, ou le nom de la variable si c'est un tableau associatif). S'il n'a pas trouvé la valeur, array_search renvoie false (comme in_array).

Syntaxe :

```
$resultat=array_search('nom_de_la_valeur', tableau) ;
```

Exemple :

```

<?php
//Recherche dans un tableau
$tableau = array("nom" => "Kanyinda", "postnom" => "Kayembe", "adresse" => "Limete");

$recherche= array_search("Kayembe", $tableau);
if($recherche==true)
{
    echo "La valeur <b>Kanyinda</b> se trouve en position <b>[<b>recherche</b>]</b><br/>";
}
else{
    print("La variable <b>Kanyinda</b> n'existe pas dans le tableau");
}
?>

```


CHAPITRE IV : BASE DE DONNEES AVEC MySQL + PHP

IV.1. Quelques notions sur MySQL

IV.1.1. Introduction

MySQL dérive directement de SQL (Structured Query Language) qui est un langage de requête vers les bases de données exploitant le modèle relationnel. Il en reprend la syntaxe mais n'en conserve pas toute la puissance puisque de nombreuses fonctionnalités de SQL n'apparaissent pas dans MySQL (sélections imbriquées, clés étrangères...)

Le serveur de base de données MySQL est très souvent utilisé avec le langage de création de pages web dynamiques : PHP.

IV.1.2. Syntaxe MySQL

IV.1.2.1. Définition de données

A. Les tables

Les tables sont les entités de base permettant de stocker les informations au sein d'une base de données. Cependant, avant de créer une table il faudra d'abord créer la base des données qui la contiendra.

- Pour créer une base de données on utilise la syntaxe suivante :

```
CREATE DATABASE nom_de_la_base;
```

nom_de_la_base désigne le nom attribué à notre base de données.

- Pour supprimer une base de données existante, on utilise l'instruction :

```
DROP DATABASE nom_de_la_base;
```

- Pour créer une table, on utilise la syntaxe:

```
CREATE TABLE nom_de_la_table
(
    nom_de_colonne1 Type_de_donnee [contrainte],
    nom_de_colonne2 Type_de_donnee [contrainte],
    ...
    nom_de_colonneN Type_de_donnee [contrainte],
);
```

- **nom_de_la_table** désigne le nom que l'on attribue à la table créée. Il doit commencer par une lettre et ne doit pas contenir plus de 254 caractères.

Les caractères blanc et /,*,-, +, « »(,)[,],{,} ;\$%^& !@#<;>...sont également interdits ;

- **nom_de_la_colonnei** désigne le nom du champ i ;
- **Type_de_donnée** désigne le type des données que doit recevoir le champ.

✓ Les types utilisés en MySQL sont :

- **Int** : entier ;
- **SmallInt** : entier signé de 16 bits (de -32768 à 32757) ;
- **Float** : nombre avec virgule flottante ;
- **Decimal(x,y)** ou **Number(x,y)** : décimal ayant x chiffres avant la virgule et y chiffres après la virgule ;
- **Real** : réel flottant codé sur 24 bits ;
- **Char (n)** : Chaîne des caractères dont la longueur fixe égale à n, avec n< 16383 ;
- **Varchar(n)** : chaîne des caractères de longueur variable, dont la taille maximale est n, avec n<16383 ;
- **Datetime** ou **Date** : date sous la forme jours/ mois/année.
- **Time** : heure sous la forme heure:minutes:secondes:tierces ;

- ✓ Contrainte désigne la contrainte qu'on attribue à la colonne ; elle n'est pas obligatoire.

Elle peut être nommée ou non.

- Pour ajouter une colonne à une table on utilise la syntaxe :

```
ALTER TABLE nom_de_la_table  
ADD Nom_de_la_colonne Type_de_donnees ;
```

- Si on réalise qu'à la création de notre table, on a déclaré une colonne qui n'est plus nécessaire ; on peut la supprimer en utilisant la syntaxe :

```
ALTER TABLE nom_de_la_table  
DROP COLUMN Nom_de_la_colonne ;
```

- Il est possible que lors de la création de la table, on déclare une colonne sous un type quelconque et que plus tard, en travaillant, on se rende compte que le type déclaré est inefficace ou qu'il occupe inutilement trop d'espace mémoire.
Pour remédier à ce problème, il faudra changer le type de cette colonne. Ceci est possible grâce à l'instruction:

```
ALTER TABLE nom_de_la_table  
MODIFY Nom_de_la_colonne Type_de_donnees ;
```

type_de_donnees est le nouveau type attribué à la colonne.

- Si une table de notre base des données ne sert plus à rien et qu'on souhaite la supprimer; il faudra utiliser l'instruction:

```
DROP TABLE Nom_de_la_table;
```

- Il est possible de ne supprimer que les données se trouvant dans une table, sans supprimer cette dernière. Pour y parvenir, il faudra utiliser l'instruction :

```
TRUNCATE TABLE Nom_de_la_table ;
```

B. Manipulations de données

B.1. Interrogation de la base de données

L'interrogation d'une base de données est l'opération la plus fréquente, voire la plus intéressante effectuée sur celle-ci. Elle est effectuée grâce à la syntaxe suivante :

```
SELECT [DISTINCT] | [ALL] attributs  
[FROM relation]  
[WHERE condition]  
[GROUP BY attributs [ASC | DESC] ]  
[HAVING condition]  
[ORDER BY attributs]  
[LIMIT [a,] b]
```

- L'option DISTINCT permet de ne sélectionner que les valeurs distinctes de la colonne (éliminer les doublons). L'option ALL permet le contraire ;
- Attributs désigne les colonnes à sélectionner. Pour sélectionner toutes les colonnes, il faut saisir le caractère * ;
- relation désigne les tables sur lesquelles s'effectue le traitement ;
- Condition est la condition de sélection;
- La clause GROUP BY permet de regrouper les lignes ayant les valeurs identiques dans une ou plusieurs colonnes issues de l'instruction SELECT, en une ligne ;
- La clause HAVING permet de restreindre la sélection des groupes figurant dans le résultat. Elle précise la condition à laquelle chaque groupe doit répondre ;
- L'instruction ORDER BY nom_de_la_colonne ASC/DESC permet de classer les valeurs de la colonne spécifiée par ordre Croissant/Décroissant.
- L'instruction LIMIT Permet de limiter le nombre de lignes du résultat.

B.2. Mise à jour des tables

- Pour ajouter des informations dans une table on utilise la syntaxe suivant :

```
INSERT INTO relation VALUES (liste exhaustive et ordonnée des valeurs)
```

- Pour modifier un ou des enregistrement(s) d'une relation, il faut préciser un critère de sélection des enregistrements à modifier (clause WHERE), il faut aussi dire quels sont les attributs dont on va modifier la valeur et quelles sont ces nouvelles valeurs (clause SET).

Sa syntaxe est :

```
UPDATE [LOW_PRIORITY] relation SET attribut=valeur, ... [WHERE condition] [LIMIT a]
```

Exemple :

```
UPDATE Personnes
SET téléphone='0156281469'
WHERE
nom='kanyinda' AND prénom = kam's'
```

Cet exemple modifie le numéro de téléphone de kanyinda kam's.

LOW_PRIORITY est une option un peu spéciale qui permet de n'appliquer la ou les modification(s) qu'une fois que plus personne n'est en train de lire dans la relation.

Il est possible de modifier les valeurs d'autant d'attributs que la relation en contient.

Exemple pour modifier plusieurs attributs :

```
UPDATE Personnes SET téléphone='0156281469',
fax=0156281812'
WHERE id = 102
```

- Pour supprimer des informations dans une table on utilise la syntaxe suivant :

```
DELETE [LOW_PRIORITY] FROM relation [ WHERE condition ] [ LIMIT a ]
```

Exemple :

```
DELETE FROM Personnes WHERE nom='Martin' AND prénom='Marc'
```

Pour vider une table de tous ces éléments, ne pas mettre de clause WHERE. Cela efface et recrée la table, au lieu de supprimer un à un chacun des tuples de la table (ce qui serait très long).

Exemple :

```
DELETE FROM Personnes
```

C. Lire des données

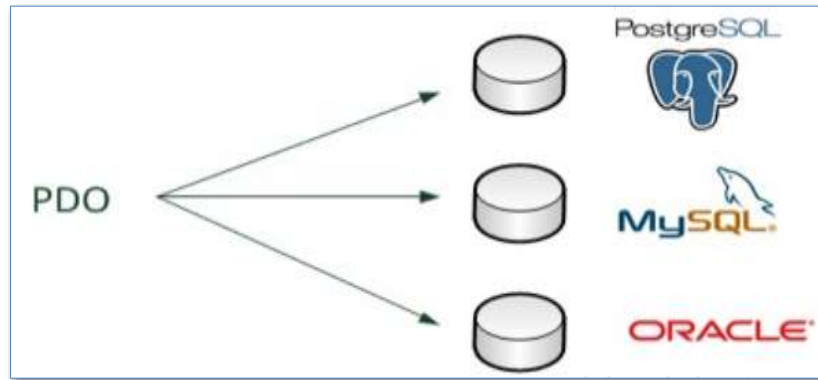
Dans cette partie, nous retournons à nos pages PHP. A partir de maintenant, nous allons apprendre à communiquer avec une base de données via PHP.

C. 1. Se connecter à la base de données en PHP

En effet, PHP propose plusieurs moyens de se connecter à une base de données MySQL.

- L'extension `mysql_` : ce sont des fonctions qui permettent d'accéder à une base de données MySQL et donc communiquer avec MySQL. Leur nom commence toujours par `mysql_`. Toutefois, ces fonctions sont vieilles et donc recommandé de plus les utiliser jusqu'à l'heure actuelle.
- L'extension `mysqli_` : ce sont des fonctions améliorées d'accès à MySQL.
- L'extension `PDO` : c'est un outil complet qui permet d'accéder à n'importe quel type de base de données. On peut donc l'utiliser pour se connecter aussi bien à MySQL que SQL Serveur, PostgreSQL ou Oracle.

Nous allons ici utiliser PDO car c'est cette méthode d'accès aux bases de données qui est la plus utilisée dans les versions actuelles de PHP. D'autre part, le gros avantage de PDO est qu'on peut l'utiliser de la même manière pour se connecter à n'importe quel autre type de base de données (PostgreSQL, Oracle...) comme illustre la figure suivante :



Noter que PDO est normalement activé par défaut quand on utilise MySQL, surtout dans PhpMyAdmin. Pour vérifier, il suffit de se rendre sur l'icône de Wamp dans la barre de tâche, faire un clic gauche dessus, puis aller dans le menu PHP/Extensions PHP et vérifier que `php_pdo_mysql` est bien coché. L'utilisation de MySQL avec PHP nécessite quatre renseignements :

- Le **nom de l'hôte** : c'est l'adresse de l'ordinateur où MySQL est installé (comme une adresse IP). Le plus souvent, MySQL est installé sur le même

ordinateur que PHP. Dans ce cas on met la valeur localhost (signifiant sur le même ordinateur)

- La **base** : c'est le nom de la base de données à laquelle on veut se connecter. Souvent créée dans le serveur MySQL (phpMyAdmin)
- Le **login** : il permet d'identifier l'utilisateur qui veut se connecter.
- Le **mot de passe**

Etant donné qu'on fait souvent des tests sur l'ordinateur chez soi, on dit qu'on travaille en **local**. Par conséquent, le nom de l'hôte sera **localhost**.

Quant au **login** et au **mot de passe**, par défaut le login est **root** et il n'y a pas de mot de passe. Vous pouvez donc définir un autre login et attribuer un mot de passe différent de vide.

Voici donc comment on doit faire pour se connecter à MySQL via PDO :

```
<?php
    $conex = new PDO('mysql:host=localhost;dbname=nom_de_BDD', 'root', '');
?>
```

Il faut reconnaître qu'il contient quelques nouveautés. En effet, PDO est ce qu'on appelle une extension **orienté objet**. C'est une façon de programmer un peu différente des fonctions classiques. La programmation Orienté Objet (POO) est prévue dans le chapitre suivant.

Si on a renseigné les bonnes informations (nom de l'hôte, de la base de données, le login et le mot de passe), rien ne devrait s'afficher dans le navigateur. Toutefois, s'il y a une erreur, PHP risque d'afficher toute la ligne qui pose le problème.

Il est préférable de gérer les erreurs. En cas de ceci, PDO renvoie ce qu'on appelle une **exception** qui permet de capturer l'erreur.

Voici comment faire pour gérer les erreurs qui peuvent survenir :

```
<?php
try
{
    $conex = new PDO('mysql:host=localhost;dbname=nom_de_BDD', 'root', '');
} catch (PDOException $ex) {
    echo $ex->getMessage();
}
?>
```

Là, PHP essaie d'exécuter les instructions à l'intérieur du bloc **try**. S'il y a une erreur, il rentre dans le bloc **catch** et fait ce qu'on lui demande.

Si au contraire tout se passe bien, PHP poursuit l'exécution du code et ne lit pas ce qu'il y a dans le bloc **catch** et la page ne devrait donc rien afficher pour le moment.

Noter que vous devrez remplacer **nom_de_BDD** par le vrai nom de votre base de données. Ceci étant le fichier de connexion est finalement :

```
<?php
try
{
    $conex = new PDO('mysql:host=localhost;dbname=gstarticle', 'root', '');
} catch (PDOException $ex) {
    echo $ex->getMessage();
}
?>
```

C. 2. Récupérer les données

Dans un premier temps, on va apprendre d'abord à lire les informations de la base de données. Ensuite viendrons les mises à jour des tables de la base de données.

Pour travailler ici, il nous faut une base de données « toute prête » qui va nous servir de support.

On se rend alors dans le phpMyAdmin pour créer la base de données ainsi que ses objets.

Une fois le serveur lancé ainsi que tous les services, voici les étapes à suivre pour la création de la base de données et ses suites :

- Aller sur l'icône de WampServer dans la barre de tâche ;
- Faire un clic gauche dessus ;

- Choisir le menu phpMyAdmin

Nous allons créer une base de données nommée **gstArticle** et seulement une table nommée **article** à titre d'exemple. Dans la zone de requête SQL saisissez ce code :

```
1 CREATE DATABASE GSTArticle;
2
3 Use GSTArticle;
4
5 CREATE TABLE Article
6 (
7     Code INT Primary Key,
8     Libelle VARCHAR(20) NOT NULL,
9     Prix DOUBLE
10 );
```

Après vous pouvez cliquer sur le bouton « **exécuter** » en bas vers la droite. S'il n'y a aucune erreur dans le script et que tout s'est bien passé, après le clic du bouton « **exécuter** » vous devez voir s'afficher la base de données récemment créée parmi les autres dans l'onglet gauche du phpMyAdmin.

! Astuce : Le champ **Code** étant un **INT**, pour faire en sorte qu'il soit auto incrément après que la table ait été créée, il suffit de taper le code SQL suivant.

```
1 ALTER TABLE article CHANGE `Code`
2 `Code` INT(11) NOT NULL AUTO_INCREMENT;
```

Sachant qu'on est sur la base de données qui contient la table bien sûr. On pourrait bien évidemment le faire au moment de la création de la table comme ceci :

```
1 CREATE TABLE article
2 (
3     Code INT Primary Key AUTO_INCREMENT,
4     ...
5 );
```

Insérons quand même quelques lignes d'enregistrements dans la table **article** pour nous s'en servir. Toujours à l'aide des scripts SQL.

Vous devez d'abord sélectionner la base de données et/ou pointer la table concernée, puis cliquez à nouveau sur le menu SQL.

```
Exécuter une ou des requêtes SQL sur la base gstarticle: ⓘ

1 INSERT INTO article (Libelle, Prix)
2     VALUES ('Ordinateur',520),
3             ('Souris',10),
4             ('Clavier',15),
5             ('Imprimante',80),
6             ('Batterie HP',100),
7             ('WebCam',50),
8             ('Flash 8Go',8),
9             ('HDD 320Go',30)
10 ;
```

Après la saisie de code, cliquer sur le bouton « **exécuter** », après quelques secondes, afficher les informations de la table toujours à l'aide de SQL :

```
1 SELECT * FROM article;
```

Résultat :

+ Options						Code	Libelle	Prix	
☐		Modifier		Copier		Effacer	1	Ordinateur	520
☐		Modifier		Copier		Effacer	2	Souris	10
☐		Modifier		Copier		Effacer	3	Clavier	15
☐		Modifier		Copier		Effacer	4	Imprimante	80
☐		Modifier		Copier		Effacer	5	Batterie HP	100
☐		Modifier		Copier		Effacer	6	WebCam	50
☐		Modifier		Copier		Effacer	7	Flash 8Go	8
☐		Modifier		Copier		Effacer	8	HDD 320Go	30

Créons maintenant une page PHP qui va nous servir d'afficher les informations contenant la table **article**. Pour récupérer des informations de la base de données, on a besoin de notre objet représentant la connexion à la base de données. Pour notre cas ici **\$conex**, et la requête de la sélection sera comme ceci :

```
$req=$conex->query("Requête SQL à taper ici");
```

On demande ainsi à effectuer une requête sur la base de données.

«**query**» en anglais signifie « requête ». On récupère ce que la base de données nous renvoyé dans un autre objet que l'on a appelé ici **\$req**.

C. 2.1. Notre première requête SQL

Effectuons la requête avec la méthode que l'on vient de découvrir :

Dans notre fichier (exemple **index.php**) que nous voulons afficher les informations, nous pouvons inclure directement le fichier contenant les instructions de connexion à la base de données supposé créé au préalable. Cette méthode permet de plus réécrire à chaque fois les instructions de connexion à la base de données dans chaque fichier .php.

Pour notre cas, le fichier s'appelle **connexionDB.php**. (*Nota : le nom de notre base de données d'exemple est **gstarticle**, à bien vérifier dans le fichier de connexion*).

D'où, nous aurons en premier lieu dans le fichier **index.php** :

```
<?php
include_once 'ConnexionDB.php';
```

Ensuite viendront les autres instructions (requête et autres).

La requête de sélection est donc :

```
<?php
$req=$conex->query("SELECT * FROM article");
?>
```

L'objet **\$req** contient quelque chose d'inexploitable. MySQL nous renvoie beaucoup d'informations qu'il faut organiser.

Pour récupérer une entrée, on prend la réponse (**\$req**) de MySQL et on y exécute **fetch()**, ce qui nous renvoie la première ligne.

```
<?php
$ligne=$req->fetch();
?>
```

fetch en anglais signifie « va chercher ».

\$ligne est un array (tableau) qui contient champ par champ les valeurs de la première entrée.

Pour afficher les informations que contient la variable **\$ligne** on aura comme instruction :

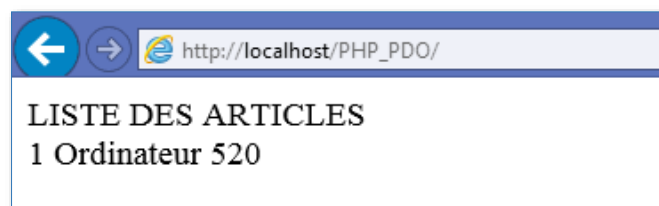
```
<?php
include_once 'ConnexionDB.php';

$req=$conex->query("SELECT * FROM article");

$ligne=$req->fetch();
echo 'LISTE DES ARTICLES <br/>';
echo $ligne['Code']." ".$ligne['Libelle']." ".$ligne['Prix']."<br/>";

?>
```

Résultat dans le navigateur :



Remarquez que nous avons qu'un article affiché, alors que nous avons plusieurs articles dans la base de données.

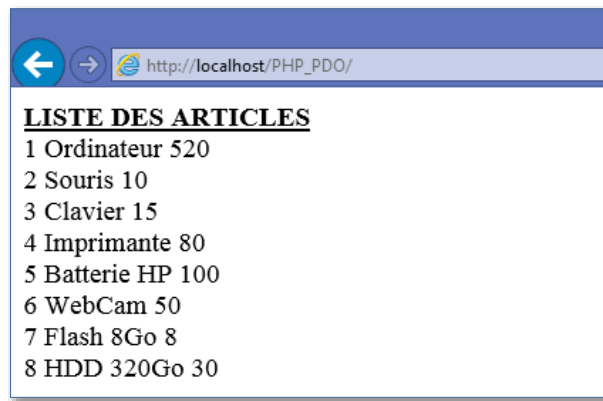
Pour résoudre ce problème, on doit recourir à une boucle qui va permettre de parcourir les entrées une à une jusqu'à la dernière. Chaque fois qu'on va appeler **\$ligne=\$req->fetch()**, on passera à l'entrée suivante. La boucle est donc répétée autant de fois qu'il y a d'entrées dans la table.

De ce fait, nous résumons tout ce qu'il nous faut pour avoir le résultat attendu comme ceci dans le fichier qui va afficher les informations (**index.php** pour notre cas ici) :

```
<?php
include_once 'ConnexionDB.php';//Inclusion du fichier ConnexionDB
$req=$conex->query("SELECT * FROM article");//Requête de la sélection
echo '<u><b>LISTE DES ARTICLES</b></u> <br/>';//Un peu de titre
while($ligne=$req->fetch())//Parcours des entrées (Enregistrements)
{
    //Affichage de chaque enregistrement de la table
    echo $ligne['Code']." ".$ligne['Libelle']." ".$ligne['Prix']."<br/>";
}
$req->closeCursor();//Fin de traitement de la requête.

?>
```

Résultat dans le navigateur :



Noter que l'instruction **\$req->closeCursor()** « fermeture du curseur d'analyse de résultat » signifie en d'autres termes qu'on doit effectuer cet appel **closeCursor()** chaque fois qu'on a fini de traiter le retour d'une requête, afin d'éviter d'avoir des problèmes à la requête suivante.

C. 2.2. Le critère de la sélection avec la clause WHERE

Grâce au mot-clé **WHERE**, on va pouvoir trier nos données.

Supposons que nous voulons afficher uniquement l'article ayant comme libelle '**Clavier**'. La requête au début sera la même qu'avant, mais on rajoutera à la fin **WHERE Libelle='Clavier'**.

Cela nous donne la requête suivante :

```
$req=$conex->query("SELECT * FROM article WHERE Libelle = 'Clavier'");
```

Si on essaye de voir de nouveau notre fichier d'affichage on a :

```
<?php
include_once 'ConnexionDB.php'; //Inclusion du fichier ConnexionDB
$req=$conex->query("SELECT * FROM article WHERE Libelle = 'Clavier'");

echo '<u><b>LISTE DES ARTICLES</b></u> <br/>'; //Un peu de titre
while($ligne=$req->fetch()) //Parcours des entrées (Enregistrements)
{
    //Affichage de chaque enregistrement de la table
    echo $ligne['Code']." ".$ligne['Libelle']." ".$ligne['Prix']."<br/>";
}
$req->closeCursor(); //Fin de traitement de la requête.
?>
```

Résultat :



Par ailleurs, il est possible de combiner plusieurs conditions dans WHERE. Par exemple, si on veut afficher tous les articles dont le libelle termine par « r » et le prix soit supérieur à « 100 ». On combinera les critères de sélection à l'aide du mot-clé **AND** (signifiant : **ET**). Cela donne ceci :

```
<?php
include_once 'ConnexionDB.php'; // Inclusion du fichier ConnexionDB
$req=$conex->query("SELECT * FROM article "
    . "WHERE Libelle like '%r' "
    . "AND Prix > 100");

echo '<u><b>LISTE DES ARTICLES</b></u> <br/>'; // Un peu de titre
while($ligne=$req->fetch()) // Parcours des entrées (Enregistrements)
{
    // Affichage de chaque enregistrement de la table
    echo $ligne['Code'] . " " . $ligne['Libelle'] . " " . $ligne['Prix'] . "<br/>";
}
$req->closeCursor(); // Fin de traitement de la requête.
?>
```

Résultat :



Un petit rappel de requête SQL :

Like opérateur SQL (signifie **comme**)

% devant la **valeur** (**r** pour cet exemple) de critère voulant dire le champ dont la valeur termine par (**r**).

% derrière la **valeur** de critère veut dire le champ dont la valeur commence par...

like	Comme 'Souris'
'Souris'	
'% S'	Terminé par 'S'
'S%'	Commence par 'S'

C. 2.3. Construire des requêtes en fonction de variables

Les requêtes que nous avons étudiées jusqu'ici étant simple et effectuant toujours la même opération. Or les choses deviennent intéressantes quand on utilise des variables de PHP dans les requêtes.

a. Concaténer une variable dans une requête : La mauvaise idée

Prenons cette requête qui récupère les articles dont le libelle égal à « Ordinateur »

```
<?php
$req=$conex->query('SELECT * FROM article WHERE Libelle = \'Ordinateur\');
?>
```

Au lieu de toujours afficher les articles dont le libelle égal à « Ordinateur », on aimerait que cette requête soit capable de s'adapter au libelle défini dans une variable, par exemple `$_GET['libelle']`. Ainsi la requête pourrait s'adapter en fonction de la demande de l'utilisateur.

On pourrait être tenté de concaténer la variable dans la requête, comme ceci :

```
<?php
$req=$conex->query('SELECT * FROM article WHERE Libelle = \''.$_GET['libelle'].'\'');
?>
```

Il est nécessaire d'entourer la chaîne de caractères d'apostrophe comme indiqué ci-dessus, d'où la présence d'antislash pour insérer les apostrophes : \'. Bien que ce code fonctionne, c'est **l'illustration parfaite de ce qu'il ne faut pas faire** et que pourtant beaucoup de gens le font encore. En effet, si la variable `$_GET['libelle']` a été modifiée par un visiteur (et nous savons à quel point il ne faut pas faire confiance à l'utilisateur), il y a un gros risque de faille de sécurité qu'on appelle **injection SQL**. Un visiteur pourrait s'amuser à insérer une requête SQL au milieu de la nôtre et potentiellement lire tout le contenu de notre base de données.

Pour utiliser un autre moyen plus sûr d'adapter nos requêtes en fonction de variable, nous utiliserons les requêtes préparées.

b. Les requêtes préparées

Le système de requêtes préparées a l'avantage d'être beaucoup plus sûr mais aussi plus rapide pour la base de données si la requête est exécutée plusieurs

fois. C'est ce qui est préconisé d'utiliser si on veut adapter une requête en fonction d'une ou plusieurs variables.

➤ Avec des marqueurs « ? »

Dans un premier temps, on prépare la requête sans sa partie variable, que l'on représentera avec un marqueur sous forme de point d'interrogation :

```
<?php
    $req=$conex->prepare('SELECT * FROM article WHERE Libelle = ?');
?>
```

Au lieu d'exécuter la requête avec **query()** comme la dernière fois, remarquez qu'ici on a appelé la méthode **prepare()** qui a comme paramètre notre requête SQL.

La requête est alors prête, sans sa partie **variable**. Maintenant, nous allons exécuter la requête en appelant la méthode **execute()** et en lui transmettant la liste de paramètres :

```
<?php
    $req = $conex->prepare('SELECT * FROM article WHERE Libelle = ?');
    $req->execute(array($_GET['libelle']));
?>
```

La requête est alors exécuter à l'aide de paramètre que l'on indique sous forme d'**array**.

S'il y a plusieurs marqueurs, il faut indiquer les paramètres dans le bon ordre.

Exemple :

```
<?php
    $req = $conex->prepare('SELECT * FROM article WHERE Libelle like ? AND Prix > ?');
    $req->execute(array($_GET['libelle'], $_GET['prix']));
?>
```

Le premier point d'interrogation de la requête sera remplacé par le contenu de la variable **\$_GET['libelle']**, et le second par le contenu de **\$_GET['prix']**. Le contenu de ces variables aura été automatiquement sécurisé pour prévenir les risques d'injection SQL.

Notre exemple résumé précédent peut alors s'écrire à l'aide de la requête prépare avec le marqueur « ? » Comme ceci :


```

<?php
//Inclusion du fichier ConnexionDB
include_once 'ConnexionDB.php';

//Définition des variables
$_GET['libelle'] = 'Ordinateur';
$_GET['prix']=100;

//Critère de sélection : La requête préparé avec le marqueur '?'
$req = $conex->prepare('SELECT * FROM article WHERE Libelle like ? AND Prix > ?');
//Execution de la requête
$req->execute(array($_GET['libelle'], $_GET['prix']));

echo '<u><b>LISTE DES ARTICLES</b></u> <br/>'; //Un peu de titre
while ($ligne = $req->fetch()) { //Parcours des entrées (Enregistrements)
    //Affichage de chaque enregistrement de la table
    echo $ligne['Code'] . " " . $ligne['Libelle'] . " " . $ligne['Prix'] . "<br/>";
}
$req->closeCursor(); //Fin de traitement de la requête.
?>

```

➤ Avec des marqueurs nominatifs

Si la requête contient beaucoup de parties variables, il peut être plus pratique de nommer les marqueurs plutôt que d'utiliser des marqueurs point d'interrogation.

Le principe est simple, pour comprendre la différence entre marqueur point d'interrogation (?) et le marqueur nominatif, dans nominatif on met après l'opérateur de tri deux point (:) suivi d'un significatif désignant le champ de critère.

Bref ! Au lieu de faire par exemple :

Marqueur point d'interrogation	Marqueur nominatif
Champ = ?	Champ = : champ

Finalement notre exemple résumé peut encore s'écrire à l'aide de requête préparée avec le marqueur nominatif comme ceci :

```

<?php
//Inclusion du fichier ConnexionDB
include_once 'ConnexionDB.php';

//Définition des variables
$_GET['libelle'] = 'Ordinateur';
$_GET['prix']=100;

//Critère de sélection : La requête préparé avec le marqueur 'nominatif'
$req = $conex->prepare('SELECT * FROM article WHERE Libelle like :libelle AND Prix > :prix');
//Execution de la requête
$req->execute(array('libelle'=>$_GET['libelle'], 'prix'=>$_GET['prix']));

echo '<u><b>LISTE DES ARTICLES</b></u> <br/>'; //Un peu de titre
while ($ligne = $req->fetch()) { //Parcours des entrées (Enregistrements)
    //Affichage de chaque enregistrement de la table
    echo $ligne['Code'] . " " . $ligne['Libelle'] . " " . $ligne['Prix'] . "<br/>";
}
$req->closeCursor(); //Fin de traitement de la requête.
?>

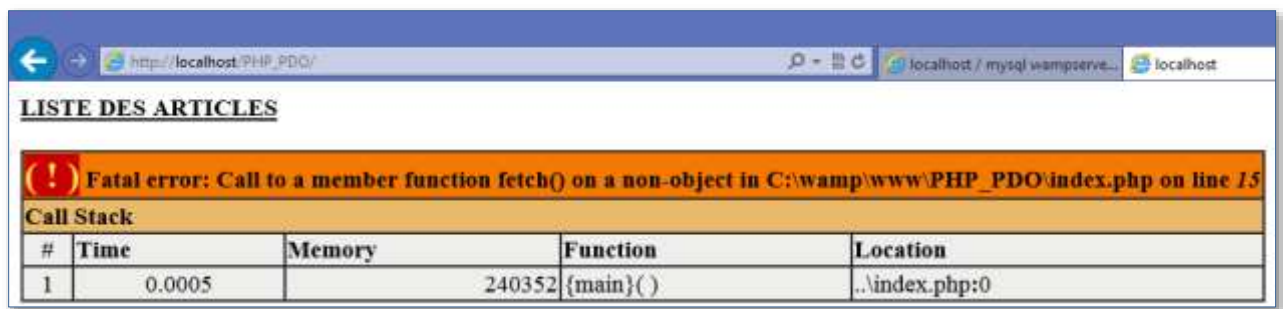
```

Cette fois-ci, ces marqueurs sont remplacés par les variables à l'aide d'un array (tableau) associatif. Quand il y a beaucoup de paramètres, cela permet parfois d'avoir plus de clarté. De plus, contrairement aux points d'interrogation, on n'est cette fois plus obligé d'envoyer les variables dans le même ordre que la requête.

C.2.4. Traquer les erreurs.

Lorsqu'une requête SQL « plante », bien souvent PHP vous dira qu'il a eu une erreur à la ligne du **fetch** :

Exemple :



Et ce n'est pas précis voire même incompréhensible. A dire vrai, ce n'est pas la ligne **fetch** qui est en cause. C'est souvent l'erreur d'avoir mal écrit la requête SQL quelque part plus haut.

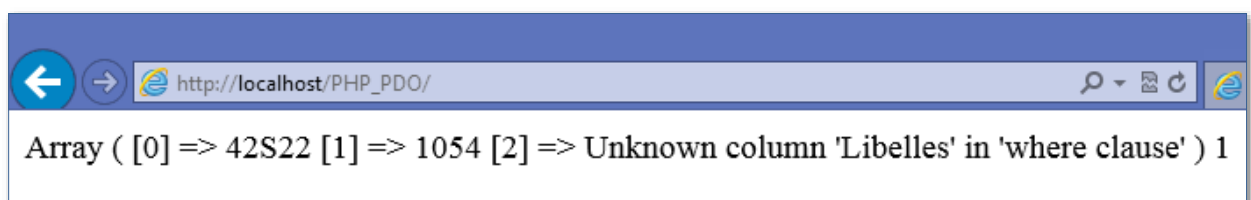
Pour éviter ce problème et surtout pour avoir un message compréhensible en cas d'erreur, il faut rajouter le code **or die (print_r(\$conex->errorInfo()))** sur la même ligne que la requête pour afficher des détails sur l'erreur.

Cas d'une requête simple.

Exemple

```
<?php
    $req = $conex->query('SELECT * FROM article WHERE Libelles=\'Ordinateur\'')
    or die(print_r($conex->errorInfo()));
?>
```

En essayant d'exécuter ces script, voici le résultat dans le navigateur :



Du coup, on déjà le problème qui se passe. Pour cet exemple, remarquez nous avons mal écrit le '**Libelles**', alors qu'il fallait écrit '**Libelle**'.

Bien que cela soit de l'anglais, mais c'est déjà beaucoup plus précis que l'erreur précédent. Si on essaye de traduire : cela signifie : « La colonne **Libelles** est introuvable dans la liste des champs » en effet, il n'y a aucun champ qui s'appelle **Libelles** dans la liste de champs de table.

Cas d'une requête préparée.

Si on utilise une requête préparée, on rajoute le code qui affiche l'erreur sur la même ligne que le **execute()**. Mais cette fois-ci, on appelle la méthode **errorInfo()** sur l'objet **\$req** et non **\$conex**.

Exemple :

```
<?php
$req = $conex->prepare('SELECT * FROM article '
    . 'WHERE Libelles like :libelle '
    . 'AND Prix > :prix');
$req->execute(array('libelle'=>$_GET['libelle'], 'prix'=>$_GET['prix']))
    or die(print_r($req->errorInfo()));
?>
```

C. 3. Mise à jour de tables.

La récupération des informations de notre base de données étant détaillée en long et large, il est maintenant temps de découvrir comment on peut ajouter, supprimer et modifier des données dans la base. Pour cela, nous allons nous servir des requêtes SQL fondamentales qu'il faut connaître : **INSERT, UPDATE et DELETE**.

C.3.1. Ajout des données (INSERT)

La requête permettant d'ajouter une entrée étant détaillée plus haut.

En voici un exemple :

```
INSERT INTO article(Libelle, Prix)
VALUES ('Scanner' 25);
```

Les (25) ici n'a pas besoin d'être entouré d'apostrophe. Seules les chaînes de caractères les nécessitent.

Dans notre table le champ **Code** est de toute façon automatiquement rempli par MySQL, il est donc inutile de le liste.

Application en PHP :

En utilisant cette requête SQL au sein d'un script PHP, cette fois-ci, au lieu de faire appel à **query()** (que l'on a utilisé dans le point précédent qui portait sur

la récupération de données), on va utiliser **exec()** qui est prévue exécuter des modifications sur la base de données.

Créons un fichier nommé **insertion.php** pour pratiquer ceci et il faut inclure le fichier de connexion à la base de données (rappelez-vous : **connexinDB.php**).

Voici l'intégralité de code d'insertion :

```
<?php
include_once './ConnexionDB.php';
$conex->exec("INSERT INTO article (Libelle, Prix) VALUES ('Scanner', 25)");

print("Article bien ajouté");

?>
```

Insertion de données variables grâce à une requête préparée.

➤ Avec des marqueurs « ? »

```
<?php
include_once './ConnexionDB.php';
$libelle='Moniteur';
$prix=95;

$req=$conex->prepare("INSERT INTO article (Libelle, Prix) VALUES(?,?)");
$req->execute(array($libelle, $prix));
print("Article bien ajouté");

?>
```

➤ Avec des marqueurs nominatifs

```

<?php
include_once './ConnexionDB.php';
$libelle='Display';
$prix=75;

$req=$conex->prepare("INSERT INTO article (Libelle, Prix) VALUES(:libelle,:prix)");
$req->execute(array(
    'libelle'=>$libelle,
    'prix'=>$prix));
print("Article bien ajouté");
?>

```

Comme pour récupération, nous avons utilisé le \$_GET, nous utilisons le \$_POST pour les données (beaucoup plus issu d'un formulaire) pour l'insertion une entrée.

Exemple :

```

<?php
include_once './ConnexionDB.php';
$_POST['libelle']='Flash USB';
$_POST['prix']=5;

$req=$conex->prepare("INSERT INTO article (Libelle, Prix) VALUES(:libelle,:prix)");
$req->execute(array(
    'libelle'=>$_POST['libelle'],
    'prix'=>$_POST['prix']));
print("Article bien ajouté");
?>

```

C.3.2. Modification des données (UPDATE)

La requête « UPDATE » permet de modifier une entrée existante déjà dans la base comme nous l'avons détaillé plus haut :

Exemple :

```

UPDATE article SET Libelle ='USB MultiPrise' WHERE Code=10;

```

Application en PHP

On crée un fichier **modification.php** qui nous servira de comprendre ceci :

```
<?php
include_once './ConnexionDB.php';

$req=$conex->exec("UPDATE article SET Libelle='USB MultiPrise' WHERE Code=10")
or die(print_r($req->errorInfo()));
echo 'Article bien modifié';
?>
```

Modification grâce aux requêtes préparées :

➤ Avec les marqueurs ?

```
<?php
include_once './ConnexionDB.php';
$_POST['code']=5;
$_POST['libelle']='Ondulaire';

$req=$conex->prepare("UPDATE article SET Libelle=? WHERE Code=?");
$req->execute(array($_POST['libelle'],$_POST['code']))
or die(print_r($req->errorInfo()));

echo 'Article bien modifié';
?>
```

➤ Avec les marqueurs nominatifs

```
<?php
include_once './ConnexionDB.php';
$_POST['code']=7;
$_POST['libelle']='Flash 16 Go';

$req=$conex->prepare("UPDATE article SET Libelle=:libelle WHERE Code=:code");
$req->execute(array('libelle'=>$_POST['libelle'],'code'=>$_POST['code']))
or die(print_r($req->errorInfo()));

echo 'Article bien modifié';
?>
```

C.3.3. Suppression des données (DELETE)

Voici une dernière requête entendue (Delete).

Exemple :

```
DELETE FROM article WHERE Code=11;
```

Application en PHP

```
<?php
include_once './ConnexionDB.php';

$req=$conex->exec("DELETE FROM article WHERE Code=11") or
die(print_r($conex->errorInfo()));
echo'Article bien supprimé';

?>
```

Suppression avec les requêtes préparées :

➤ Avec les marqueurs ?

```
<?php
include_once './ConnexionDB.php';

$_POST['code']=9;
$req=$conex->prepare("DELETE FROM article WHERE Code=?") ;
$req->execute(array($_POST['code']))
or die(print_r($req->errorInfo()));
echo'Article bien supprimé';

?>
```

➤ Avec les marqueurs nominatifs

```
<?php
include_once './ConnexionDB.php';

$_POST['code']=3;
$req=$conex->prepare("DELETE FROM article WHERE Code=:code") ;
$req->execute(array('code'=>$_POST['code']))
or die(print_r($req->errorInfo()));
echo'Article bien supprimé';

?>
```

D. La programmation Orientée Objet (POO)

Il y a nombreuses façons de programmer en PHP. C'est ce qui fait sa force. On peut en effet commencer à créer ses premières pages basiques en PHP avec très peu de connaissance au départ, mais il est possible de programmer avec

des outils avancés, plus complexes mais aussi plus solides et plus flexibles. La programmation orientée objet est une technique de programmation célèbre qui existe depuis des années maintenant. PHP ne l'a pas inventée, d'autres langages comme le C++, C#, Java, Python l'utilisaient bien évidemment. La POO ne va pas améliorer subitement la qualité de votre site comme par magie. En revanche, elle va vous aider à mieux organiser votre code, à le préparer à des futures évolutions et à rendre certaines portions réutilisables pour gagner en temps et en clarté.

Bibliographie

- [1] M. Nebra, Apprendre à créer votre site web avec HTML5 et CSS3, Licence Creative Commons 6. 2.0, 2013.
- [2] O. Fauvel –Jaeger, Introduction à l'animation en HTML5 et JavaScript
- [3] Nod_, Bonnes pratiques JavaScript, Licence Creative Commons 7 2.0, 2012.
- [4] M. Nebra, Concevez votre site web avec PHP et MySQL Creative Commons 6. 2.0, 2012.
- [5] V. Thuillier, Programmer en Orienté Objet en PHP, Creative Commons 6. 2.0, 2012.
- [6] E. Daspet, C. Pierre de Geyer, PHP 5 Avancé 4^{ème} Edit, EYROLLES
- [7] C. Gribaumont, Administrez vos bases de données avec MySQL Creative Commons 6. 2.0, 2012.
- [5] S. Duchateau L'accessibilité Web Creative Commons 6. 2.0, 2013.
- [6] D. Alex, jQuery, dynamisez vos page plus simplement Creative Commons 6. 2.0, 2013.
- [7] Sainior, Découvrez la puissance de jQuery UI, Creative Commons 6. 2.0, 2013.
- [8] Vincent1870, Adopter un style de programmation clair avec le modèle MVC, Creative Commons 7. 2.0, 2011.